

GUÍA PRÁCTICA · ARQUITECTURA DE SOFTWARE

Guía completa de microservicios

De la teoría a la práctica con Spring Boot y WebClient

Por Chantal · @chanti.dev

Octubre 2026 · Java 21 · Spring Boot 3.x / 4.x · Spring Cloud

Índice

Parte I — Fundamentos de los microservicios 4

1. Qué son los microservicios 5

2. Para qué se usan y cuándo aplicarlos 7

3. Cómo funcionan: componentes de la arquitectura 9

4. Diseño: límites, dominio y descomposición 11

5. Comunicación entre servicios 13

6. Datos y transacciones distribuidas 15

7. Catálogo de patrones de microservicios 17

8. Escalabilidad 19

9. Fiabilidad y resiliencia 22

10. Observabilidad 24

11. Seguridad en microservicios 26

12. Testing en microservicios 27

13. Despliegue y DevOps 28

14. Buenas prácticas y antipatrones 30

Parte II — Microservicios con Spring Boot 32

15. El ecosistema Spring para microservicios 33

16. Tu primer microservicio con Spring Boot 35

17. Un ejemplo completo: tienda con varios servicios 40

18. Service Discovery con Eureka y balanceo de carga 41

19. Configuración centralizada 43

20. API Gateway con Spring Cloud Gateway 45

21. Comunicación síncrona con WebClient 48

22. Resiliencia con Resilience4j 55

23. Mensajería asíncrona con Kafka 58

24. Datos, transacciones y sagas con Spring 60

25. Observabilidad con Spring Boot 62

26. Seguridad con Spring Security y OAuth2 64

27. Testing de microservicios con Spring Boot 66

28. Contenedores y despliegue de servicios Spring Boot 69

29. Escalabilidad y rendimiento en Spring Boot 72

30. Checklist final, hoja de ruta y glosario 75

PARTE I

Fundamentos de los microservicios

Qué son, para qué sirven, cómo funcionan, cuándo aplicarlos y qué patrones los hacen viables. Esta parte no depende de ningún lenguaje ni framework.

1. Qué son los microservicios

Un microservicio es un servicio pequeño, autónomo y desplegable de forma independiente que implementa una capacidad concreta del negocio. Una arquitectura de microservicios construye una aplicación como un conjunto de esos servicios, que se comunican entre sí a través de la red mediante contratos explícitos (APIs o eventos).

La definición clásica de James Lewis y Martin Fowler (2014) resume las características comunes: servicios componentizados, organizados alrededor de capacidades de negocio, con infraestructura automatizada, gobierno descentralizado, datos descentralizados y diseño tolerante al fallo.

Características que definen un microservicio

- **Responsabilidad única y acotada.** Cada servicio resuelve una capacidad de negocio (pedidos, pagos, catálogo, notificaciones), no una capa técnica.
- **Despliegue independiente.** Puedes publicar una nueva versión de un servicio sin recompilar ni redesplegar el resto.
- **Datos propios.** Cada servicio es dueño de su base de datos o esquema. Ningún otro servicio lee ni escribe en esas tablas.
- **Comunicación por contratos.** Los servicios se hablan por HTTP/REST, gRPC o mensajería, nunca compartiendo código interno ni base de datos.
- **Equipos autónomos.** Un equipo pequeño puede ser dueño de uno o varios servicios de extremo a extremo: diseño, código, despliegue y operación.
- **Tecnología libre pero acotada.** En teoría cada servicio puede usar su propio lenguaje o base de datos; en la práctica conviene limitar la variedad.
- **Diseñados para fallar.** Asumen que la red y otros servicios fallarán, y se protegen con timeouts, reintentos y cortacircuitos.

Monolito frente a microservicios

Un monolito empaqueta toda la aplicación en una sola unidad de despliegue con una única base de datos. No es un antipatrón: es la opción correcta para la mayoría de proyectos que empiezan. La diferencia es dónde viven los límites entre módulos: en un monolito son límites de código (paquetes, módulos); en microservicios son límites de red y de despliegue.

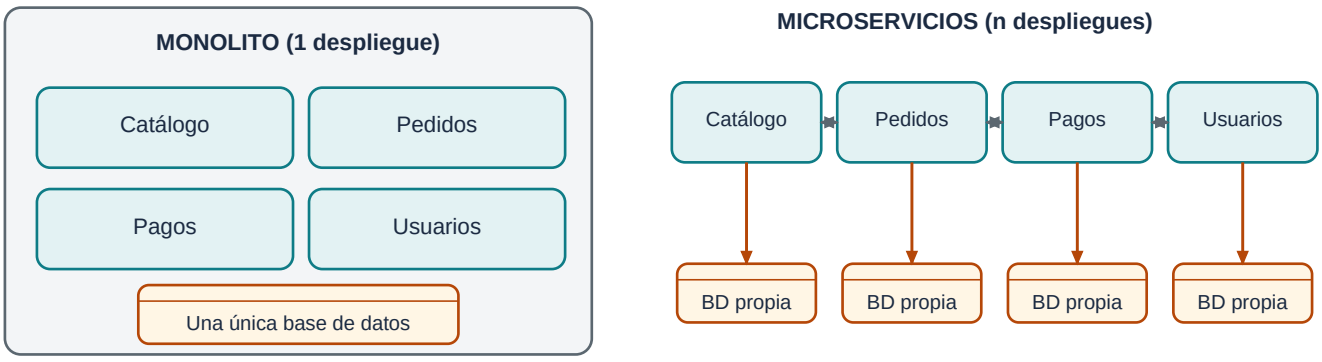


Figura 1.1 · En el monolito los módulos comparten proceso y base de datos; en microservicios cada servicio es un proceso con sus propios datos.

Aspecto	Monolito	Microservicios
Despliegue	Una sola unidad. Cualquier cambio exige redesplegar todo.	Cada servicio se despliega por separado y con su propio ritmo.
Escalado	Se escala la aplicación entera, aunque solo un módulo lo necesite.	Se escala solo el servicio que lo necesita.
Base de datos	Normalmente una compartida; transacciones ACID triviales.	Una por servicio; consistencia eventual y sagas entre servicios.
Complejidad	Concentrada en el código.	Desplazada a la red, la operación y la coordinación entre servicios.
Rendimiento entre módulos	Llamadas en memoria (nanosegundos).	Llamadas por red (milisegundos) que pueden fallar.
Equipos	Un equipo o varios coordinados sobre el mismo código.	Equipos autónomos con propiedad de sus servicios.
Pruebas	Más sencillas de ejecutar de extremo a extremo.	Requieren pruebas de contrato y entornos más complejos.
Observabilidad	Un log, un proceso, un depurador.	Exige logs centralizados, métricas y trazado distribuido.
Coste inicial	Bajo.	Alto: CI/CD, contenedores, orquestación, monitorización.

Ventajas

- **Escalado selectivo:** pagas recursos solo donde hay carga.
- **Despliegues frecuentes y de bajo riesgo:** un cambio pequeño afecta a un servicio pequeño.
- **Aislamiento de fallos:** si un servicio secundario cae, el resto puede seguir funcionando de forma degradada.
- **Autonomía de equipos:** menos coordinación para publicar; ownership claro.
- **Evolución tecnológica gradual:** puedes reescribir o actualizar un servicio sin migrar toda la aplicación.
- **Alineación con el negocio:** los servicios siguen los límites del dominio.

Inconvenientes

- **Complejidad distribuida:** latencia, fallos parciales, timeouts, orden de mensajes, duplicados.
- **Consistencia de datos:** no hay transacciones ACID entre servicios; hay que diseñar sagas y compensaciones.
- **Sobrecarga operativa:** muchos pipelines, imágenes, configuraciones, certificados y alertas.
- **Depuración más difícil:** una petición atraviesa varios procesos; sin trazado distribuido es opaca.
- **Coste de infraestructura y de personas:** requiere cultura DevOps y madurez del equipo.
- **Riesgo de monolito distribuido:** servicios tan acoplados que deben desplegarse juntos, con todos los problemas de ambos mundos.

El término se popularizó entre 2011 y 2014 a partir de la experiencia de Netflix, Amazon y Spotify. Se apoya en Domain-Driven Design, la entrega continua, los contenedores y Kubernetes. Conviene recordar la ley de Conway: la estructura del software acaba reflejando la estructura de comunicación de la organización que lo construye.

2. Para qué se usan y cuándo aplicarlos

Casos de uso típicos

- **Plataformas de comercio electrónico:** catálogo, carrito, pedidos, pagos, envíos e inventario con requisitos de carga muy distintos.
- **Servicios financieros y fintech:** separación estricta de pagos, cuentas, antifraude y notificaciones, con requisitos de auditoría.
- **Streaming y medios:** recomendaciones, reproducción, facturación y perfiles escalan de forma independiente.
- **SaaS multi-equipo:** módulos con ciclos de vida distintos y varios equipos trabajando a la vez.
- **Sistemas con picos de carga localizados:** por ejemplo, el servicio de búsqueda o de checkout en campañas.
- **Migración de sistemas legados:** extraer capacidades poco a poco con el patrón Strangler Fig.
- **Backends de IA y agentes:** servicios de orquestación, recuperación de documentos y herramientas que escalan por separado y se exponen por API.

Cuándo tiene sentido adoptarlos

- La aplicación ya es **grande** y el monolito frena los cambios (builds largos, despliegues arriesgados, conflictos constantes).
- **Varios equipos** trabajan a la vez y se bloquean mutuamente en el mismo código base.
- Partes concretas necesitan **escalar o desplegarse por separado**, con ritmos o requisitos distintos.
- Existen **requisitos de aislamiento** (seguridad, cumplimiento normativo, fallos que no deben propagarse).
- El dominio está **bien entendido** y se identifican límites estables entre capacidades.
- La organización tiene **madurez DevOps**: CI/CD, contenedores, observabilidad y guardias operativas.

Cuándo no conviene

- Proyecto nuevo, **dominio todavía difuso**: los límites que traces hoy serán incorrectos mañana y moverlos entre servicios es caro.
- **Equipo pequeño** (por ejemplo, menos de 5-8 personas): el coste operativo supera el beneficio.
- No hay automatización de despliegue ni monitorización fiable.
- El problema es de **rendimiento puntual** que se resuelve con caché, índices o escalado vertical.
- Se busca microservicios por moda o por currículum, no por una necesidad medible.

> **BUENA PRÁCTICA · MONOLITO PRIMERO**
> Empieza con un **monolito modular**: un único despliegue pero con módulos de dominio bien separados, interfaces internas explícitas y sin acceso cruzado a tablas. Si más adelante un módulo necesita vivir por su cuenta, extraerlo es un trabajo acotado. Esta estrategia se conoce como *monolith first* (Martin Fowler).

Preguntas para decidir

Pregunta	Si la respuesta es sí…
¿Podemos desplegar un cambio pequeño en minutos y sin miedo?	Si no, el problema puede ser de CI/CD y pruebas, no de arquitectura.
¿Hay equipos que se pisan constantemente en el mismo código?	Señal a favor de separar por dominio.
¿Una parte necesita escalar 10 veces más que el resto?	Señal a favor de extraerla.
¿Conocemos los límites del dominio?	Si no, aplaza la división y refina el modelo.
¿Tenemos observabilidad, automatización y guardias?	Si no, los microservicios multiplicarán el dolor.
¿Aceptamos consistencia eventual en los flujos entre dominios?	Si el negocio exige transacciones fuertes entre estas partes, mantenlas juntas.

3. Cómo funcionan: componentes de la arquitectura

Un sistema de microservicios es mucho más que los servicios en sí. Para que funcione hacen falta piezas de plataforma que resuelven problemas que en un monolito no existían: cómo se encuentran los servicios, cómo entra el tráfico, dónde se guarda la configuración o cómo se sigue una petición a través de varios procesos.

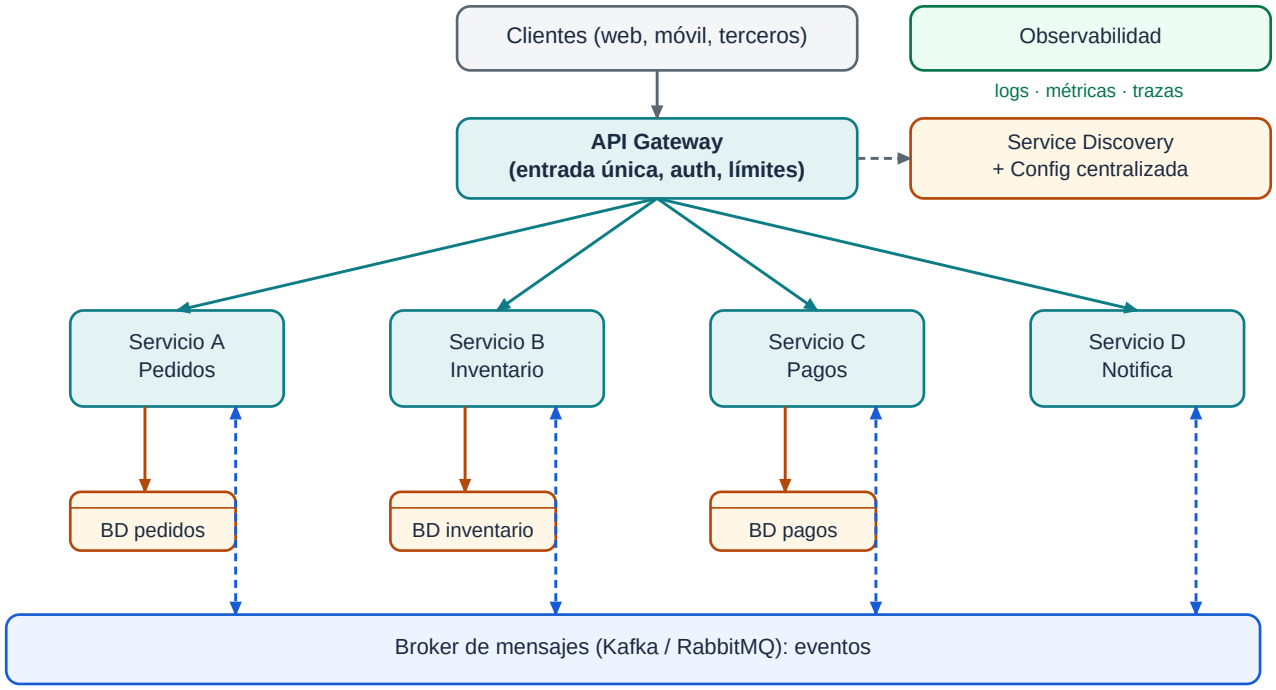


Figura 3.1 · Arquitectura de referencia: gateway, servicios con datos propios, mensajería y servicios de plataforma.

Componentes principales

Componente	Qué resuelve	Ejemplos
API Gateway	Punto de entrada único. Enruta, autentica, aplica límites de tasa, agrega respuestas y oculta la topología interna.	Spring Cloud Gateway, Kong, NGINX, Envoy, AWS API Gateway
Service Discovery	Permite que un servicio encuentre las instancias vivas de otro sin direcciones fijas.	Eureka, Consul, DNS de Kubernetes
Balanceo de carga	Reparte las peticiones entre las instancias de un servicio.	Spring Cloud LoadBalancer, Kubernetes Service, Envoy
Configuración centralizada	Externaliza la configuración por entorno y permite cambiarla sin recompilar.	Spring Cloud Config, Consul, ConfigMaps/Secrets
Broker de mensajes	Comunicación asíncrona y desacoplada mediante eventos y colas.	Kafka, RabbitMQ, SNS/SQS, Pulsar
Persistencia por servicio	Datos propios del servicio, con el motor que mejor encaje.	PostgreSQL, MongoDB, Redis, Elasticsearch
Observabilidad	Ver qué ocurre: logs, métricas y trazas correlacionadas.	OpenTelemetry, Prometheus, Grafana, Loki, Jaeger, Tempo
Seguridad	Identidad, autorización, cifrado entre servicios y gestión de secretos.	Keycloak, OAuth2/OIDC, mTLS, Vault
Orquestación y despliegue	Ejecutar, escalar y recuperar los contenedores.	Docker, Kubernetes, Helm, Argo CD
CI/CD	Construir, probar y desplegar cada servicio de forma automática.	GitHub Actions, GitLab CI, Jenkins

Recorrido de una petición

1. El cliente envía `POST /api/orders` al **API Gateway**.
2. El gateway valida el token, aplica el límite de tasa y consulta el **registro de servicios** para localizar una instancia sana del servicio de pedidos.
3. El **servicio de pedidos** valida la petición, guarda el pedido en **su base de datos** y consulta por API síncrona al servicio de inventario si hay stock.
4. Si todo va bien, publica un evento `OrderCreated` en el **broker**. No espera a nadie.
5. El servicio de **pagos** y el de **notificaciones** consumen el evento por su cuenta y actúan cuando les toca.
6. Cada paso emite logs, métricas y un tramo de traza con el mismo identificador, de modo que toda la petición se puede reconstruir.

4. Diseño: límites, dominio y descomposición

El error más caro en microservicios es dibujar mal los límites. Un servicio demasiado fino genera conversaciones constantes por red; uno demasiado grueso es un monolito pequeño. La referencia más útil para acertar es Domain-Driven Design.

Bounded context

Un *bounded context* es la frontera dentro de la cual un modelo y su lenguaje tienen un significado único. La palabra «Cliente» no significa lo mismo en Ventas (alguien que compra), en Facturación (una entidad fiscal) y en Soporte (alguien con incidencias). Cada contexto debería ser un candidato natural a servicio, con su propio modelo y sus propios datos.

Cómo dividir

- **Por capacidad de negocio:** qué hace la empresa (gestionar pedidos, cobrar, enviar).
- **Por subdominio (DDD):** núcleo, de soporte y genérico. Invierte más en servicios del núcleo.
- **Por volatilidad:** separa lo que cambia con frecuencia de lo estable.
- **Por escala y requisitos no funcionales:** lo que necesita otra disponibilidad, seguridad o carga.
- **Por equipo:** un servicio que un equipo puede entender y operar completo.

Qué tamaño debe tener un servicio

No hay un número de líneas correcto. Heurísticas útiles: un equipo pequeño puede entenderlo entero; puede reescribirse en semanas, no en años; cambia por un único motivo de negocio; y rara vez necesita desplegarse junto a otro servicio. Si dos servicios siempre cambian y se despliegan juntos, probablemente son uno.

> **ANTIPATRÓN · MICROSERVICIOS NANOSCÓPICOS**
> **Servicios por entidad (CRUD de tablas).** Crear un servicio «Cliente», otro «Dirección» y otro «Teléfono» produce un modelo anémico, decenas de llamadas por operación y ninguna cohesión de negocio.

Migrar un monolito: Strangler Fig

En lugar de reescribirlo todo, se coloca una fachada (gateway) delante del monolito y se redirige capacidad por capacidad hacia servicios nuevos. El monolito se va vaciando hasta que desaparece o queda reducido a un núcleo estable.

1. Identifica una capacidad con límites claros y valor claro (por ejemplo, notificaciones o catálogo).
2. Pon un gateway o proxy delante del monolito.
3. Construye el nuevo servicio y desvía una parte del tráfico (canary).
4. Sincroniza los datos si hace falta (CDC, eventos) y valida paridad.
5. Retira el código antiguo del monolito y repite con la siguiente capacidad.

Capa anticorrupción

Cuando un servicio nuevo debe convivir con un sistema legado o de terceros, una *anti-corruption layer* traduce entre sus modelos, de modo que el modelo del servicio nuevo no quede contaminado por conceptos ajenos.

5. Comunicación entre servicios

Síncrona: petición y respuesta

El llamante espera la respuesta. Es sencilla de razonar, pero acopla disponibilidad y latencia: si el servicio B está lento, A también lo está.

- **REST/HTTP + JSON:** estándar, simple, legible, ideal para APIs públicas y operaciones CRUD.
- **gRPC:** contratos con Protobuf, binario, HTTP/2, streaming bidireccional y menor latencia. Muy buena opción para tráfico interno entre servicios.
- **GraphQL:** útil en el borde (gateway/BFF) para que el cliente elija los campos que necesita.

Asíncrona: eventos y mensajes

El emisor publica un mensaje y continúa. Los consumidores lo procesan cuando pueden. Reduce el acoplamiento temporal y absorbe picos, a cambio de consistencia eventual y mayor dificultad para seguir el flujo.

Tipo	Descripción	Ejemplo
Evento	Notifica algo que **ya ocurrió**. El emisor no sabe quién lo consume.	`OrderCreated`, `PaymentFailed`
Comando	Pide a un servicio concreto que **haga** algo.	`ReserveStock`, `SendEmail`
Consulta asíncrona	Petición con respuesta diferida por otra cola.	Request/reply sobre RabbitMQ

Criterio	Síncrona (REST/gRPC)	Asíncrona (broker)
Acoplamiento temporal	Alto: ambos deben estar disponibles.	Bajo: el consumidor puede estar caído y recuperar después.
Latencia percibida	Suma de las llamadas encadenadas.	El emisor responde enseguida; el trabajo ocurre después.
Consistencia	Más inmediata.	Eventual.
Depuración	Más sencilla.	Requiere trazado y correlación de mensajes.
Cuándo usarla	Consultas y operaciones que necesitan respuesta inmediata.	Notificaciones, integración entre dominios, flujos largos, picos.

Brokers: Kafka frente a RabbitMQ

Aspecto	Apache Kafka	RabbitMQ
Modelo	Log distribuido y persistente, particionado.	Cola de mensajes con exchanges y routing.
Retención	Los mensajes permanecen según política; se pueden releer.	Se eliminan al consumirse (salvo streams).
Orden	Garantizado por partición (clave).	Por cola.
Rendimiento	Muy alto, orientado a streaming y grandes volúmenes.	Alto, orientado a mensajería flexible.
Casos típicos	Event sourcing, analítica, pipelines, CDC.	Tareas, colas de trabajo, routing complejo.

Contratos y versionado

- Define el contrato **primero** (OpenAPI, Protobuf, AsyncAPI, JSON Schema) y trátalo como producto.
- Aplica la **ley de Postel** con cabeza: tolera campos desconocidos al leer, sé estricto al escribir.
- Los cambios **compatibles hacia atrás** (añadir campos opcionales) no requieren nueva versión; los incompatibles sí (`/v2`, nuevo tópico).
- Usa **contract testing** (Pact, Spring Cloud Contract) para detectar roturas antes de producción.
- Mantén un **schema registry** si usas Avro o Protobuf sobre Kafka.

Idempotencia

Los reintentos y la entrega *at-least-once* de los brokers provocan duplicados. Una operación es idempotente si ejecutarla varias veces produce el mismo resultado que una sola. Se logra con claves de idempotencia (`Idempotency-Key` en HTTP), restricciones únicas en base de datos y una tabla de mensajes ya procesados en los consumidores.

> **IDEA CLAVE**
> En un sistema distribuido, «exactamente una vez» es en la práctica «al menos una vez + consumidores idempotentes».

6. Datos y transacciones distribuidas

Base de datos por servicio

Cada servicio es dueño de sus datos. Si necesita información de otro, la pide por API o la recibe por eventos y guarda una copia local de lo que necesita (una vista desnormalizada). Esto permite elegir el motor adecuado (relacional, documental, clave-valor, búsqueda) y evolucionar el esquema sin coordinar con nadie.

> **ANTIPATRÓN · SHARED DATABASE**
> **Base de datos compartida.** Dos servicios que leen las mismas tablas están acoplados por el esquema: cambiar una columna rompe a ambos, y no se pueden desplegar de forma independiente. Es la forma más habitual de construir un monolito distribuido.

Una base de datos por servicio no implica un servidor por servicio. Puedes compartir el servidor físico siempre que cada servicio use su propio esquema o base de datos con credenciales independientes y sin acceso cruzado.

Consistencia eventual

Sin transacciones ACID entre servicios, el sistema pasa por estados intermedios y converge a un estado consistente. El negocio debe aceptarlo: un pedido puede estar «pendiente de pago» durante unos segundos. Diseña la interfaz y los procesos para reflejar esos estados.

Saga

Una saga divide una transacción de negocio en una secuencia de transacciones locales. Si un paso falla, se ejecutan **transacciones compensatorias** que deshacen los pasos anteriores (por ejemplo, liberar el stock reservado).

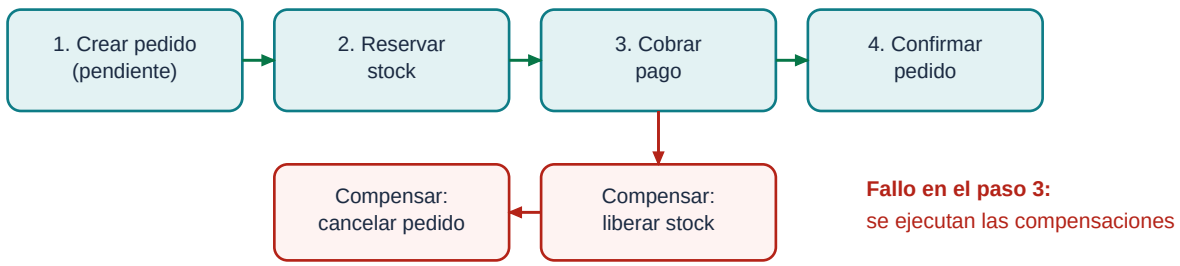


Figura 6.1 · Saga con compensaciones: si el cobro falla, se libera el stock y se cancela el pedido.

Variante	Cómo coordina	Ventajas	Inconvenientes
Coreografía	Cada servicio reacciona a eventos de otros y emite los suyos. No hay coordinador.	Sencilla al inicio, desacoplada, sin punto central.	El flujo queda repartido; difícil de seguir cuando crece.
Orquestación	Un orquestador envía comandos y decide el siguiente paso.	Flujo explícito, fácil de entender y monitorizar.	Introduce un componente central que mantener.

Herramientas habituales para orquestar: Temporal, Camunda, Netflix Conductor, AWS Step Functions, o un orquestador propio con una máquina de estados persistida.

Outbox transaccional

Problema: guardar en la base de datos y publicar en el broker son dos operaciones, y no pueden ser atómicas. Si la base se actualiza y el broker falla, el resto del sistema no se entera. Solución: dentro de la misma transacción local se guarda el cambio **y** el evento en una tabla `outbox`. Un proceso aparte (poller o CDC con Debezium) lee esa tabla y publica los eventos, reintentando hasta lograrlo.

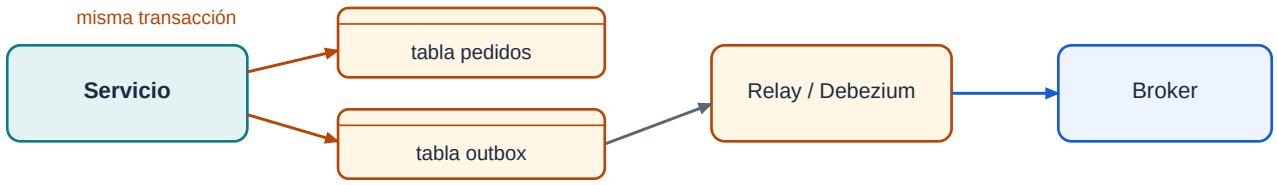


Figura 6.2 · Patrón Transactional Outbox.

CQRS y Event Sourcing

CQRS (Command Query Responsibility Segregation) separa el modelo de escritura del de lectura. Las lecturas usan vistas optimizadas y desnormalizadas, alimentadas por eventos, que pueden vivir en otra base de datos (por ejemplo, Elasticsearch). Es útil cuando las lecturas y las escrituras tienen necesidades muy distintas.

Event Sourcing guarda como fuente de verdad la secuencia de eventos que han ocurrido en lugar del estado actual. El estado se reconstruye reproduciendo los eventos. Aporta auditoría completa y viaje en el tiempo, pero añade bastante complejidad (versionado de eventos, snapshots, consultas). No lo adoptes salvo que haya una razón clara.

Consultas que cruzan servicios

- **API Composition:** un servicio (o el gateway/BFF) llama a varios y compone la respuesta.
- **Vista materializada local:** el servicio consume eventos y mantiene una copia de los datos ajenos que necesita.
- **CQRS con proyecciones:** una base de lectura alimentada por eventos de varios servicios.

7. Catálogo de patrones de microservicios

Resumen de los patrones más usados, agrupados por el problema que resuelven. Los detalles de implementación con Spring Boot aparecen en la parte II.

Estructura y descomposición

Patrón	Problema	Idea
Decompose by business capability / subdomain	Cómo dividir el sistema.	Un servicio por capacidad de negocio o bounded context.
Strangler Fig	Migrar un monolito sin reescribirlo.	Fachada que desvía tráfico gradualmente a servicios nuevos.
Anti-Corruption Layer	Integrar con legado o terceros sin contaminar el modelo.	Capa de traducción entre modelos.
Database per Service	Acoplamiento por datos.	Cada servicio es dueño de su almacenamiento.

Entrada y descubrimiento

Patrón	Problema	Idea
API Gateway	Los clientes no deben conocer la topología interna.	Punto de entrada único con enrutado, seguridad y límites.
Backend for Frontend (BFF)	Cada cliente (web, móvil) necesita datos distintos.	Un gateway específico por tipo de cliente.
Service Discovery	Las instancias cambian de dirección continuamente.	Registro dinámico (cliente o servidor) de instancias.
Externalized Configuration	Configuración distinta por entorno sin recompilar.	Config server, variables de entorno, ConfigMaps.
Sidecar / Service Mesh	Aplicar mTLS, retries y telemetría sin tocar el código.	Proxy junto a cada servicio (Istio, Linkerd).

Resiliencia

Patrón	Problema	Idea
Timeout	Esperas infinitas que agotan hilos.	Limitar siempre el tiempo máximo de cada llamada.
Retry con backoff y jitter	Fallos transitorios.	Reintentar con esperas crecientes y aleatorias, solo operaciones idempotentes.
Circuit Breaker	Un servicio caído arrastra a sus llamantes.	Corta las llamadas tras demasiados fallos y prueba de nuevo después.
Bulkhead	Un servicio lento consume todos los recursos.	Aislar pools de hilos o conexiones por dependencia.
Rate Limiter	Sobrecarga por un cliente o ráfaga.	Limitar peticiones por unidad de tiempo.
Fallback	Degradar con elegancia.	Respuesta alternativa (caché, valor por defecto) cuando falla la dependencia.
Health Check API	Saber si una instancia puede recibir tráfico.	Endpoints de liveness y readiness.

Datos y consistencia

Patrón	Problema	Idea
Saga	Transacciones entre servicios.	Secuencia de transacciones locales con compensaciones.
Transactional Outbox	Guardar y publicar de forma atómica.	Tabla outbox + relay o CDC.
Idempotent Consumer	Mensajes duplicados.	Registrar mensajes procesados y descartar repetidos.
CQRS	Lecturas y escrituras con necesidades distintas.	Modelos separados, proyecciones alimentadas por eventos.
Event Sourcing	Auditoría e historial completo.	Guardar eventos como fuente de verdad.
API Composition	Consultas entre servicios.	Componer respuestas llamando a varios servicios.

Observabilidad y operación

Patrón	Problema	Idea
Log Aggregation	Logs dispersos en muchas instancias.	Centralizar en un almacén consultable (Loki, ELK).
Distributed Tracing	Seguir una petición entre servicios.	Identificadores de traza propagados (OpenTelemetry).
Application Metrics	Medir salud y rendimiento.	Métricas RED/USE expuestas a Prometheus.
Correlation ID	Relacionar logs de varios servicios.	ID único propagado en cabeceras y logs.
Deployment strategies	Reducir el riesgo de publicar.	Blue-green, canary, rolling, feature flags.

8. Escalabilidad

Escalar es la capacidad de atender más carga añadiendo recursos sin degradar el servicio. En microservicios la escalabilidad es uno de los argumentos principales, pero no ocurre sola: depende de cómo diseñes los servicios, los datos y la infraestructura.

Vertical y horizontal

	Escalado vertical (scale up)	Escalado horizontal (scale out)
Qué hace	Más CPU o memoria en la misma máquina.	Más instancias del servicio detrás de un balanceador.
Límite	El techo físico de la máquina.	Prácticamente ilimitado si el servicio es *stateless*.
Disponibilidad	Punto único de fallo.	Tolera la caída de instancias.
Coste	Crece de forma no lineal.	Crece de forma lineal; se puede ajustar a la demanda.
Complejidad	Baja.	Requiere balanceo, discovery y servicios sin estado.

Requisitos para escalar horizontalmente

- **Servicios sin estado (stateless):** ninguna instancia guarda sesión ni datos que otra necesite. El estado vive en la base de datos, en una caché compartida (Redis) o en el token del cliente (JWT).
- **Arranque rápido y apagado limpio:** permiten añadir y quitar instancias con rapidez sin perder peticiones.
- **Configuración externa y entorno reproducible:** una instancia nueva debe comportarse igual que las demás.
- **Balanceo de carga:** repartir peticiones entre instancias sanas (client-side con LoadBalancer o server-side con Kubernetes/Envoy).
- **Health checks:** el balanceador solo debe enviar tráfico a instancias *ready*.

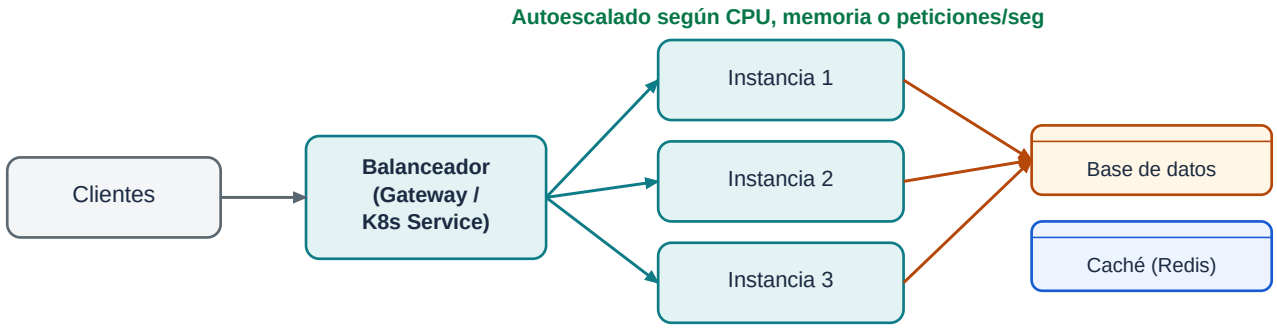


Figura 8.1 · Escalado horizontal: varias réplicas sin estado detrás de un balanceador, con el estado en servicios externos.

Palancas de escalabilidad, por capas

1. Aplicación

- **Estado externo:** sesiones en Redis o tokens JWT; nada de estado en memoria compartido entre peticiones.
- **Caché:** de lecturas costosas o frecuentes (cache-aside con TTL). La invalidación es la parte difícil: define quién invalida y cuándo.
- **Paginación y límites:** nunca devuelvas colecciones sin acotar.
- **Trabajo asíncrono:** mueve a colas todo lo que no necesite respuesta inmediata (emails, informes, integraciones).
- **Paralelismo controlado:** llamadas independientes en paralelo con límites de concurrencia.
- **Backpressure:** no aceptar más trabajo del que se puede procesar (límites de cola, rate limiting, reactive streams).

2. Datos

- **Índices adecuados** y consultas revisadas antes de escalar el hardware.
- **Réplicas de lectura:** separa lecturas de escrituras cuando el patrón de acceso lo permite.
- **Particionado (sharding):** divide los datos por clave (cliente, región) cuando una sola base no basta.
- **Pools de conexiones dimensionados:** el total de conexiones de todas las réplicas no debe superar el máximo de la base de datos.
- **Motor adecuado:** búsqueda en Elasticsearch, series temporales en una base especializada, caché en Redis.
- **CQRS:** modelos de lectura optimizados y escalables de forma independiente.

3. Mensajería

- **Particiones y consumidores en grupo:** en Kafka, el paralelismo máximo de un grupo es el número de particiones.
- **Colas para absorber picos:** el productor escribe rápido y los consumidores procesan a su ritmo.
- **Dead Letter Queue:** aparta mensajes que fallan repetidamente sin bloquear el resto.

4. Infraestructura

- **Autoescalado (HPA en Kubernetes)** basado en CPU, memoria o métricas personalizadas (peticiones/seg, lag de Kafka con KEDA).
- **Límites y peticiones de recursos** definidos por servicio para que el planificador reparta bien.
- **CDN y caché en el borde** para contenido estático y respuestas cacheables.
- **Multi-zona o multi-región** para disponibilidad y latencia.

> **CUIDADO**
> Escalar un servicio no sirve de nada si su dependencia es el cuello de botella. Si 20 réplicas atacan a una base de datos que soporta 100 conexiones, solo conseguirás tirarla antes. Mide siempre dónde está el límite real.

Cómo escalar bien: método

1. **Define objetivos** medibles: latencia p95/p99, peticiones por segundo, disponibilidad.
2. **Mide** con pruebas de carga realistas (k6, Gatling, JMeter) y observabilidad.

3. **Encuentra el cuello de botella** (CPU, memoria, pool de conexiones, base de datos, red, servicio externo).
4. **Arréglalo en la capa más barata:** consulta o índice, caché, asincronía, y solo después hardware.
5. **Automatiza** el escalado y valida con pruebas de carga y de caos.
6. **Repite:** el cuello de botella se desplaza al siguiente punto débil.

Capacidad y costes

Calcula cuánta capacidad necesita cada servicio con la ley de Little: *concurrencia* = *tasa de llegada* × *latencia media*. Si un servicio recibe 200 peticiones por segundo con una latencia media de 100 ms, mantiene unas 20 peticiones en vuelo a la vez. Dimensiona hilos, conexiones y réplicas con ese orden de magnitud y deja margen para picos.

9. Fiabilidad y resiliencia

En un sistema distribuido, el fallo es la norma. La red pierde paquetes, los servicios tardan, las instancias se reinician. La resiliencia consiste en que esos fallos no se conviertan en caídas generales.

Los fallos típicos

- **Latencia alta:** un servicio responde, pero lento. Suele ser peor que un fallo limpio porque agota hilos y conexiones de los llamantes.
- **Fallos transitorios:** timeouts puntuales, reinicios, conmutaciones.
- **Fallos en cascada:** un servicio caído arrastra a los que dependen de él, y a los que dependen de ellos.
- **Tormenta de reintentos:** muchos clientes reintentan a la vez y rematan al servicio que se recupera.
- **Datos duplicados o desordenados** por reintentos y entrega at-least-once.

Timeouts

Toda llamada de red debe tener un timeout, y debe ser menor que el del llamante. Define timeout de conexión y de lectura/respuesta por separado, y un presupuesto total de tiempo por petición. Sin timeouts, un servicio lento puede bloquear todos tus hilos.

Reintentos (retry)

- Reintenta solo errores **transitorios** (timeouts, 502/503/504), nunca errores de validación (4xx).
- Reintenta solo operaciones **idempotentes**, o que lleven una clave de idempotencia.
- Usa **backoff exponencial con jitter** para no sincronizar a los clientes.
- Limita el número de intentos (2-3) y el tiempo total.
- Cuidado con los reintentos en capas: si tres capas reintentan 3 veces, una llamada se convierte en 27.

Circuit Breaker

Funciona como un fusible. Monitoriza las llamadas a una dependencia y, si la tasa de fallos o de lentitud supera un umbral, **abre el circuito**: durante un tiempo las llamadas fallan inmediatamente (o usan un *fallback*) sin intentar siquiera conectar. Pasado ese tiempo pasa a **semiabierto** y deja pasar unas pocas llamadas de prueba: si funcionan, vuelve a cerrarse.

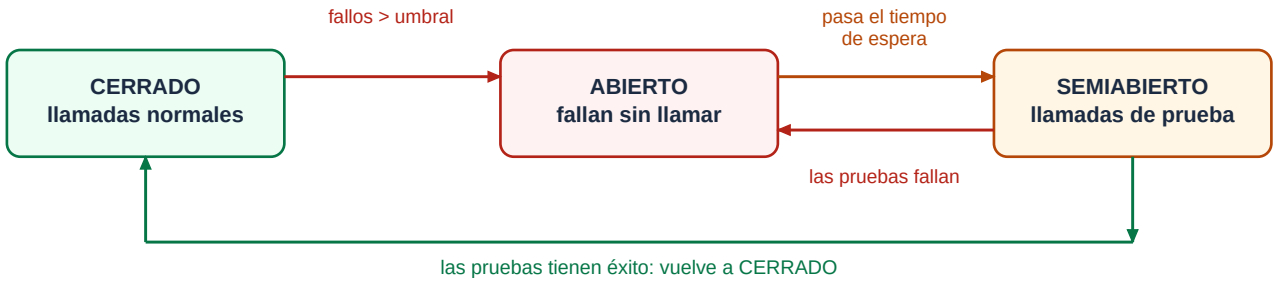


Figura 9.1 · Estados de un Circuit Breaker.

Bulkhead (mamparos)

Como los compartimentos de un barco, aísla recursos por dependencia: un pool de hilos o un número máximo de llamadas concurrentes para cada servicio externo. Si una dependencia se bloquea, solo agota su compartimento y el resto del servicio sigue funcionando.

Fallback y degradación elegante

Cuando una dependencia no responde, decide qué ofrecer: datos de caché, un valor por defecto, una versión reducida de la funcionalidad o un error claro. Es mejor un catálogo sin recomendaciones que una página de error. Un fallback debe ser **simple y fiable**: si el propio fallback puede fallar, no sirve.

Otras defensas

- **Rate limiting y load shedding:** rechaza trabajo cuando estás saturado para proteger lo que ya está en curso.
- **Health checks:** *liveness* (¿está vivo el proceso?) y *readiness* (¿puede recibir tráfico?). Distínguelos: reiniciar por un fallo de la base de datos compartida puede empeorar el incidente.
- **Graceful shutdown:** al recibir la señal de parada, dejar de aceptar peticiones y terminar las que están en curso.
- **Redundancia:** al menos 2-3 réplicas en zonas distintas para servicios críticos.
- **Chaos engineering:** inyecta fallos de forma controlada (Chaos Monkey, Litmus, Chaos Mesh) para comprobar que las defensas funcionan.

SLI, SLO y SLA

Término	Significado	Ejemplo
SLI	Indicador medible de calidad de servicio.	Porcentaje de peticiones con éxito; latencia p99.
SLO	Objetivo interno sobre un SLI.	99,9 % de peticiones correctas en 30 días.
SLA	Compromiso contractual con consecuencias.	Reembolso si la disponibilidad baja del 99,5 %.
Error budget	Margen de fallo permitido por el SLO.	0,1 % de 30 días ≈ 43 minutos de indisponibilidad.

10. Observabilidad

Observabilidad es poder entender el estado interno del sistema a partir de lo que emite. En microservicios es imprescindible: sin ella, depurar un fallo repartido entre diez servicios es adivinar.

Los tres pilares

Pilar	Qué responde	Buenas prácticas	Herramientas
Logs	¿Qué pasó exactamente?	Estructurados (JSON), con nivel, traceId y correlationId; centralizados; sin datos sensibles.	Loki, ELK/OpenSearch, Datadog
Métricas	¿Cómo se comporta el sistema en el tiempo?	Métodos RED y USE; etiquetas con cardinalidad controlada; alertas sobre síntomas.	Prometheus, Grafana, Micrometer
Trazas	¿Por dónde pasó la petición y dónde se fue el tiempo?	Propagación de contexto (W3C Trace Context); muestreo razonable.	OpenTelemetry, Jaeger, Tempo, Zipkin

Métricas que importan

- **RED (servicios):** *Rate* (peticiones/s), *Errors* (tasa de error), *Duration* (latencia, en percentiles p50/p95/p99).
- **USE (recursos):** *Utilization*, *Saturation*, *Errors* de CPU, memoria, disco, red, pools.
- **Golden signals (Google SRE):** latencia, tráfico, errores y saturación.
- **Negocio:** pedidos por minuto, pagos fallidos, tiempo de entrega.
- **Mensajería:** *lag* de consumidores, mensajes en DLQ, tiempo en cola.

Trazado distribuido

Cada petición recibe un **trace ID** al entrar al sistema. Cada operación (llamada HTTP, consulta SQL, publicación de mensaje) genera un **span** con su duración y su padre. El contexto se propaga por cabeceras (`traceparent`) y por los mensajes. Con ello puedes ver el árbol completo de una petición y localizar el servicio que más tarda.

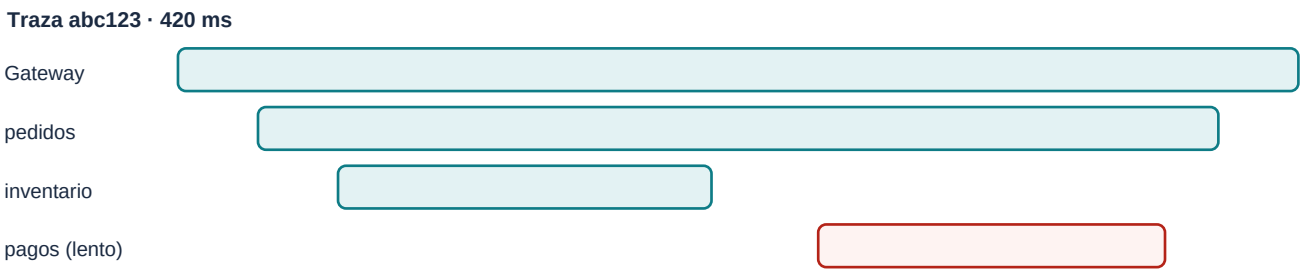


Figura 10.1 · Cascada de spans: el tramo más largo señala el cuello de botella.

Alertas

- Alerta sobre **síntomas** que afectan al usuario (errores, latencia), no sobre causas (CPU alta puede ser normal).
- Cada alerta debe ser **accionable** y llevar un enlace al *runbook*.

- Usa alertas por **consumo del error budget** (burn rate) en lugar de umbrales fijos.
- Revisa y elimina alertas ruidosas: la fatiga de alertas acaba con la guardia.

> **BUENA PRÁCTICA**
> Genera el correlation/trace ID en el borde (gateway), propágalo en todas las llamadas y mensajes, y escríbelo en cada línea de log. Es la mejora de depuración con mejor relación esfuerzo/beneficio.

11. Seguridad en microservicios

La superficie de ataque crece con cada servicio y cada llamada de red. La seguridad debe pensarse en capas y sin asumir que la red interna es de confianza (*zero trust*).

Autenticación y autorización

- **OAuth 2.0 + OpenID Connect:** un proveedor de identidad (Keycloak, Auth0, Entra ID, Cognito) autentica al usuario y emite tokens.
- **JWT:** token firmado que viaja en `Authorization: Bearer`. Cada servicio valida firma, emisor, audiencia y caducidad.
- **Gateway como punto de control:** valida el token y aplica políticas gruesas; cada servicio sigue aplicando la autorización fina.
- **Scopes y roles:** permisos por operación (`orders:read`, `orders:write`) y por recurso (el usuario solo ve sus pedidos).
- **Cliente a servicio:** *client credentials* para llamadas máquina a máquina. Para propagar la identidad del usuario, reenvía su token o usa *token exchange*.

Protección en tránsito y en reposo

- TLS en el borde y **mTLS** entre servicios (service mesh o certificados gestionados).
- Cifrado de datos sensibles en reposo y de las copias de seguridad.
- **Gestión de secretos** con Vault, AWS/GCP Secret Manager o Kubernetes Secrets cifrados; nunca en el repositorio ni en la imagen.
- Rotación periódica de credenciales y claves.

Defensa en profundidad

- **Validación de entrada** en cada servicio, no solo en el gateway.
- **Mínimo privilegio:** cuentas de servicio y de base de datos con los permisos justos.
- **Políticas de red** (NetworkPolicies) para limitar qué servicio habla con cuál.
- **Escaneo de dependencias e imágenes** (Dependabot, Trivy, Snyk) y SBOM.
- **Límites de tasa y protección frente a abuso** en el gateway.
- **Auditoría:** registra accesos y cambios relevantes, sin guardar datos sensibles en claro.
- Sigue el **OWASP API Security Top 10** como lista de comprobación.

12. Testing en microservicios

La pirámide adaptada

Nivel	Qué prueba	Velocidad	Notas
Unitarias	Lógica de dominio y clases aisladas.	Muy rápida	La base de la pirámide; sin red ni base de datos.
Integración (por servicio)	Repositorios, controladores, clientes HTTP, mensajería, con dependencias reales o simuladas.	Rápida/media	Testcontainers para BD y broker reales; WireMock para otros servicios.
Contrato	Que consumidor y proveedor respetan el mismo contrato de API o de evento.	Rápida	Pact, Spring Cloud Contract. Evita la mayoría de pruebas end-to-end.
Componente	Un servicio completo con sus dependencias externas simuladas.	Media	Arranca el servicio entero en un contenedor.
End-to-end	Flujos críticos atravesando varios servicios reales.	Lenta	Pocas y estables; solo los caminos de negocio esenciales.

Contract testing

Evita que un cambio en el proveedor rompa a sus consumidores sin enterarse. Con *consumer-driven contracts*, cada consumidor declara qué espera (campos, códigos, formato) y el proveedor verifica en su CI que sigue cumpliéndolo. Es mucho más barato y fiable que levantar todo el sistema para probarlo.

Pruebas de resiliencia y de rendimiento

- **Pruebas de carga** (k6, Gatling, JMeter) con tráfico realista y entornos comparables a producción.
- **Pruebas de fallo:** simula latencia, errores y caídas de dependencias (WireMock, Toxiproxy) y comprueba timeouts, reintentos y *fallbacks*.
- **Chaos engineering** en entornos controlados.
- **Pruebas en producción de forma segura:** canary, feature flags, monitorización sintética.

13. Despliegue y DevOps

Contenedores y orquestación

Cada servicio se empaqueta como una imagen de contenedor inmutable, que se promociona sin cambios entre entornos. Kubernetes es el orquestador de facto: programa los contenedores, los escala, los reinicia, hace discovery y balanceo, y gestiona configuración y secretos.

Recurso de Kubernetes	Para qué sirve
Deployment	Mantiene N réplicas de un servicio y gestiona las actualizaciones progresivas.
Service	Dirección estable y balanceo entre las réplicas (discovery vía DNS).
Ingress / Gateway API	Entrada de tráfico externo y enrutado HTTP.
ConfigMap / Secret	Configuración y secretos inyectados como variables o ficheros.
HPA / KEDA	Autoescalado por métricas de CPU, memoria o eventos.
Probes	Liveness, readiness y startup para decidir reinicios y tráfico.
PodDisruptionBudget	Garantiza un mínimo de réplicas durante mantenimientos.

CI/CD por servicio

1. Cada servicio tiene su **pipeline independiente**: compilar, probar, análisis estático, escaneo de seguridad.
2. Se construye una **imagen versionada** (por commit o por etiqueta) y se publica en un registro.
3. Despliegue **automático** a desarrollo y a preproducción; a producción con aprobación o de forma continua.
4. **Infraestructura como código** (Terraform, Helm, Kustomize) y GitOps (Argo CD, Flux): el repositorio es la fuente de verdad.
5. **Rollback** rápido y comprobado.

Estrategias de despliegue

Estrategia	Cómo funciona	Cuándo usarla
Rolling update	Sustituye instancias de una en una.	Por defecto en Kubernetes; cambios compatibles.
Blue-green	Dos entornos completos; se conmuta el tráfico de uno a otro.	Cambios de riesgo medio; rollback inmediato.
Canary	Un porcentaje pequeño del tráfico va a la nueva versión mientras se vigilan métricas.	Cambios de riesgo alto; validar con tráfico real.
Feature flags	El código se despliega apagado y se activa por usuario o porcentaje.	Separar despliegue de lanzamiento.

Las doce factores (12-factor app)

Un conjunto de principios para aplicaciones nativas de la nube que encajan muy bien con microservicios.

Factor	Idea
1. Código base	Un repositorio por servicio desplegable, rastreado en control de versiones.
2. Dependencias	Declaradas explícitamente y aisladas (Maven/Gradle).
3. Configuración	En el entorno, no en el código.
4. Servicios de apoyo	BD, broker o caché tratados como recursos conectables por URL.
5. Build, release, run	Etapas separadas; artefactos inmutables.
6. Procesos	Sin estado y sin compartir nada; el estado va a servicios de apoyo.
7. Asignación de puertos	El servicio se exporta escuchando en un puerto.
8. Concurrencia	Se escala creando más procesos (instancias).
9. Desechabilidad	Arranque rápido y apagado limpio.
10. Paridad dev/prod	Entornos lo más parecidos posible.
11. Logs	Flujo de eventos a stdout; la plataforma los recoge.
12. Procesos de administración	Tareas puntuales como procesos de una sola ejecución (jobs).

14. Buenas prácticas y antipatrones

Buenas prácticas

Diseño

- Alinea los servicios con **bounded contexts**, no con tablas ni capas técnicas.
- Prefiere **bajo acoplamiento y alta cohesión**: lo que cambia junto vive junto.
- Diseña **APIs orientadas a casos de uso**, estables y versionadas; evita exponer el modelo interno.
- Prefiere **eventos** para integrar dominios y **API síncrona** para consultas que necesitan respuesta inmediata.
- Evita cadenas largas de llamadas síncronas: cuantos más saltos, menor disponibilidad conjunta.
- Empieza **más grueso** y divide cuando haya una razón medible.

Datos

- Base de datos **por servicio**, sin acceso cruzado.
- Acepta la **consistencia eventual** y diséñala: estados intermedios, compensaciones, idempotencia.
- Usa **outbox** para publicar eventos con garantías.
- Versiona los esquemas y las migraciones (Flyway, Liquibase) como parte del servicio.

Resiliencia

- **Timeouts en todas las llamadas.** Sin excepciones.
- Reintentos con **backoff y jitter**, solo en operaciones idempotentes.
- **Circuit breaker, bulkhead y fallbacks** en las dependencias críticas.
- Diseña para la **degradación elegante**.
- Prueba los fallos: no asumas que tus defensas funcionan.

Operación

- **Observabilidad desde el primer día:** logs estructurados, métricas RED y trazas.
- **Automatiza todo:** pruebas, build, despliegue, infraestructura.
- Cada servicio tiene un **propietario claro** (equipo, canal, runbook).
- Define **SLOs** y alerta sobre ellos.
- Documenta con **OpenAPI/AsyncAPI**, ADRs (decisiones de arquitectura) y diagramas C4.
- Mantén una **plataforma interna** (plantillas de servicio, librerías, pipelines) para reducir el coste de crear servicios nuevos.

Antipatrones frecuentes

Antipatrón	Síntoma	Cómo evitarlo
Monolito distribuido	Los servicios se despliegan siempre juntos; un cambio toca varios.	Revisa los límites; fusiona servicios acoplados; usa contratos estables.
Base de datos compartida	Varios servicios leen o escriben las mismas tablas.	Una BD por servicio; comunícate por API o eventos.
Servicios nanoscópicos	Cientos de servicios CRUD diminutos; operaciones con decenas de llamadas.	Agrupa por capacidad de negocio.
Llamadas síncronas en cadena	A llama a B, que llama a C, que llama a D.	Eventos, composición en el borde, replicar datos de lectura.
Sin timeouts ni reintentos prudentes	Cascadas de fallos y tormentas de reintentos.	Timeouts, backoff con jitter, circuit breaker.
Falta de observabilidad	«Algo falla y no sabemos dónde».	Trazas, logs correlacionados, métricas RED.
Librería compartida con lógica de negocio	Actualizar una librería obliga a redesplegar todos los servicios.	Comparte solo utilidades técnicas estables; el negocio se queda en cada servicio.
Versionado inexistente	Un cambio de API rompe a los consumidores.	Compatibilidad hacia atrás; versiones; contract tests.
Adopción prematura	Microservicios en un proyecto pequeño sin necesidad.	Monolito modular primero.
Orquestador omnisciente	Un servicio central con toda la lógica; los demás son CRUD.	Distribuye responsabilidades por dominio.

Lista de comprobación antes de ir a producción

- Cada servicio tiene **health checks** (liveness y readiness), **timeouts** configurados y **límites de recursos**.
- Hay **logs estructurados, métricas y trazas** correlacionados, y paneles de RED por servicio.
- Existen **alertas accionables** con runbook y un propietario de guardia.
- Las APIs están **documentadas y versionadas**, y hay contract tests en CI.
- Las **migraciones de BD** son compatibles hacia atrás (expand/contract).
- Los **secretos** no están en el código ni en las imágenes; hay rotación.
- Se ha probado **el escalado y la recuperación** (carga, caída de instancias, caída de una dependencia).
- Hay un **plan de rollback** y copias de seguridad con restauración probada.
- El **coste** está estimado y monitorizado.

PARTE II

Microservicios con Spring Boot

Del servicio individual a la arquitectura completa: Spring Cloud, Gateway, WebClient, Resilience4j, mensajería, observabilidad, seguridad, testing y despliegue.

15. El ecosistema Spring para microservicios

Spring Boot es hoy la opción más extendida para construir microservicios en Java. Aporta arranque rápido, configuración por convención, servidor embebido y un ecosistema muy completo. Spring Cloud añade las piezas específicas de sistemas distribuidos: gateway, discovery, configuración, balanceo y cortacircuitos.

Piezas y para qué sirven

Pieza	Función	Dependencia / proyecto
Spring Boot	Base: autoconfiguración, servidor embebido, Actuator, empaquetado.	`spring-boot-starter-*`
Spring Web MVC	API REST con modelo bloqueante (hilo por petición). Compatible con hilos virtuales.	`spring-boot-starter-web`
Spring WebFlux	Modelo reactivo no bloqueante. Incluye `WebClient`.	`spring-boot-starter-webflux`
Spring Data JPA	Persistencia relacional con repositorios.	`spring-boot-starter-data-jpa`
Spring Cloud Gateway	API Gateway reactivo: rutas, filtros, límites, circuit breaker.	Spring Cloud Gateway
Eureka	Registro y descubrimiento de servicios.	`spring-cloud-starter-netflix-eureka-*`
Spring Cloud LoadBalancer	Balanceo del lado del cliente (`lb://servicio`).	`spring-cloud-starter-loadbalancer`
Spring Cloud Config	Configuración centralizada y versionada en Git.	Config Server / Client
Resilience4j	Circuit breaker, retry, bulkhead, rate limiter, time limiter.	`resilience4j-spring-boot3`
Spring Cloud Stream / Spring for Apache Kafka	Mensajería con Kafka o RabbitMQ.	`spring-kafka` o Stream binders
Micrometer + OpenTelemetry	Métricas y trazado distribuido.	`micrometer-tracing-bridge-otel`
Spring Security	OAuth2/OIDC, resource server, JWT.	`spring-boot-starter-oauth2-resource-server`
Testcontainers / WireMock	Pruebas de integración realistas.	`spring-boot-testcontainers`

Versiones y compatibilidad

Spring Cloud se publica por «trenes» (por ejemplo, 2024.0.x, 2025.0.x, 2025.1.x), y cada tren es compatible con una versión concreta de Spring Boot. Usa siempre el BOM `spring-cloud-dependencies` y consulta la matriz de compatibilidad oficial en spring.io/projects/spring-cloud antes de combinar versiones.

NOTA · VERSIONES
Los ejemplos de esta guía están escritos para **Java 21** y **Spring Boot 3.x**, y la mayoría funciona igual en Spring Boot 4. Revisa la guía de migración de la versión que uses: pueden cambiar nombres de starters, paquetes de las anotaciones de test y algunos prefijos de propiedades. Donde hay diferencias conocidas lo indico en el texto.

Con Spring Cloud o con Kubernetes

Parte de Spring Cloud resuelve problemas que Kubernetes ya cubre: discovery (Service + DNS), balanceo (Service), configuración (ConfigMap/Secret) y, con una malla, mTLS y reintentos. Una regla práctica:

- **Despliegue en Kubernetes:** usa el discovery y la configuración de la plataforma y conserva de Spring Cloud el Gateway, Resilience4j y la observabilidad.
- **Sin orquestador (máquinas virtuales, Docker Compose, entorno de aprendizaje):** Eureka y Config Server siguen siendo útiles.

16. Tu primer microservicio con Spring Boot

Estructura del proyecto

Organiza el código por **dominio y capa**, no solo por tipo técnico. Una estructura hexagonal ligera funciona muy bien y mantiene el dominio libre de detalles de infraestructura.

Estructura recomendada

```
order-service/
├── pom.xml
└── src/main/
    ├── java/com/acme/orders/
    │   ├── OrderServiceApplication.java
    │   ├── api/                      # adaptadores de entrada
    │   │   ├── OrderController.java
    │   │   ├── dto/ (CreateOrderRequest, OrderResponse)
    │   │   └── ApiExceptionHandler.java
    │   ├── application/              # casos de uso
    │   │   └── OrderService.java
    │   ├── domain/                   # modelo y reglas de negocio
    │   │   ├── Order.java
    │   │   └── OrderStatus.java
    │   └── infrastructure/           # adaptadores de salida
    │       ├── persistence/ (OrderRepository)
    │       ├── client/ (InventoryClient)   # WebClient
    │       └── messaging/ (OrderEventPublisher)
    └── resources/
        ├── application.yml
        └── db/migration/V1__create_orders.sql
```

Dependencias (pom.xml)

pom.xml (fragmento)

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>3.5.x</version>   <!-- usa la última 3.5.x o la 4.x que elijas -->
</parent>

<properties>
  <java.version>21</java.version>
  <spring-cloud.version>2025.0.x</spring-cloud.version> <!-- según la matriz -->
</properties>

<dependencies>
  <dependency><groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId></dependency>
  <dependency><groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId></dependency>
  <dependency><groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId></dependency>
  <dependency><groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId></dependency>
  <dependency><groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-webflux</artifactId></dependency> <!-- WebClient -->
  <dependency><groupId>org.postgresql</groupId>
    <artifactId>postgresql</artifactId><scope>runtime</scope></dependency>
  <dependency><groupId>org.flywaydb</groupId>
    <artifactId>flyway-core</artifactId></dependency>
</dependencies>

<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>org.springframework.cloud</groupId>
```

```
      <artifactId>spring-cloud-dependencies</artifactId>
      <version>${spring-cloud.version}</version>
      <type>pom</type><scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>
```

> **NOTA**
> Al incluir `spring-boot-starter-web` y `spring-boot-starter-webflux` a la vez, la aplicación arranca como **MVC (servlet)** y `WebClient` se usa como cliente. Si quieres una aplicación totalmente reactiva, deja solo WebFlux.

Configuración básica (application.yml)

application.yml

```
spring:
  application:
    name: order-service
  datasource:
    url: jdbc:postgresql://localhost:5432/orders
    username: ${DB_USER:orders}
    password: ${DB_PASSWORD:orders}
    hikari:
      maximum-pool-size: 10
      connection-timeout: 3000
  jpa:
    open-in-view: false           # evita consultas perezosas fuera de la transacción
    hibernate.ddl-auto: validate  # el esquema lo gestiona Flyway
  mvc:
    problemdetails:
      enabled: true               # errores en formato RFC 9457 (ProblemDetail)
  threads:
    virtual:
      enabled: true               # Java 21+: hilos virtuales para peticiones

server:
  port: 8081
  shutdown: graceful              # termina las peticiones en curso al parar
spring.lifecycle.timeout-per-shutdown-phase: 20s

management:
  endpoints.web.exposure.include: health,info,prometheus
  endpoint.health.probes.enabled: true
```

Dominio, DTOs y repositorio

Order.java y DTOs

```
@Entity
@Table(name = "orders")
public class Order {
    @Id @GeneratedValue(strategy = GenerationType.UUID)
    private UUID id;
    private String customerId;
    @Enumerated(EnumType.STRING)
    private OrderStatus status;
    private BigDecimal total;
    private Instant createdAt = Instant.now();

    protected Order() {}                       // requerido por JPA
    public Order(String customerId, BigDecimal total) {
        this.customerId = customerId;
        this.total = total;
        this.status = OrderStatus.PENDING;
    }
    public void confirm() { this.status = OrderStatus.CONFIRMED; }
```

```
    public void cancel()  { this.status = OrderStatus.CANCELLED; }
    // getters...
}

public enum OrderStatus { PENDING, CONFIRMED, CANCELLED }

public record CreateOrderRequest(
        @NotBlank String customerId,
        @NotEmpty List<@Valid Item> items) {
    public record Item(@NotBlank String sku, @Min(1) int quantity,
                       @NotNull @Positive BigDecimal unitPrice) {}
}

public record OrderResponse(UUID id, OrderStatus status, BigDecimal total) {}

public interface OrderRepository extends JpaRepository<Order, UUID> {
    Page<Order> findByCustomerId(String customerId, Pageable pageable);
}
```

Los **records** de Java son ideales para DTOs: inmutables, concisos y compatibles con Jackson y Bean Validation. No expongas nunca las entidades JPA directamente en la API.

Controlador y servicio

OrderController.java

```
@RestController
@RequestMapping("/api/orders")
public class OrderController {
    private final OrderService service;
    public OrderController(OrderService service) { this.service = service; }

    @PostMapping
    public ResponseEntity<OrderResponse> create(
            @RequestHeader("Idempotency-Key") String key,
            @Valid @RequestBody CreateOrderRequest request) {
        OrderResponse created = service.placeOrder(key, request);
        return ResponseEntity.created(URI.create("/api/orders/" + created.id()))
                             .body(created);
    }

    @GetMapping("/{id}")
    public OrderResponse get(@PathVariable UUID id) {
        return service.get(id);
    }

    @GetMapping
    public Page<OrderResponse> list(@RequestParam String customerId, Pageable pageable) {
        return service.listByCustomer(customerId, pageable);
    }
}
```

OrderService.java

```
@Service
public class OrderService {
    private final OrderRepository repository;
    private final InventoryClient inventory;      // cliente WebClient (capítulo 21)
    private final OrderEventPublisher events;

    public OrderService(OrderRepository repository, InventoryClient inventory,
                        OrderEventPublisher events) {
        this.repository = repository;
        this.inventory = inventory;
        this.events = events;
    }

    @Transactional
    public OrderResponse placeOrder(String idempotencyKey, CreateOrderRequest req) {
        // 1) comprobar y reservar stock en el servicio de inventario (síncrono)
        inventory.reserve(idempotencyKey, req.items());
```

```
        // 2) persistir el pedido
        BigDecimal total = req.items().stream()
                .map(i -> i.unitPrice().multiply(BigDecimal.valueOf(i.quantity())))
                .reduce(BigDecimal.ZERO, BigDecimal::add);
        Order order = repository.save(new Order(req.customerId(), total));

        // 3) publicar el evento (idealmente con outbox, capítulo 23)
        events.orderCreated(order);
        return new OrderResponse(order.getId(), order.getStatus(), order.getTotal());
    }

    @Transactional(readOnly = true)
    public OrderResponse get(UUID id) {
        return repository.findById(id)
                .map(o -> new OrderResponse(o.getId(), o.getStatus(), o.getTotal()))
                .orElseThrow(() -> new OrderNotFoundException(id));
    }
    // listByCustomer(...)
}
```

Manejo de errores con ProblemDetail

Spring 6 incorpora `ProblemDetail`, que implementa el estándar RFC 9457 (antes 7807) para errores HTTP. Un formato de error único y predecible facilita mucho el trabajo de los consumidores.

ApiExceptionHandler.java

```
@RestControllerAdvice
public class ApiExceptionHandler extends ResponseEntityExceptionHandler {

    @ExceptionHandler(OrderNotFoundException.class)
    ProblemDetail notFound(OrderNotFoundException ex) {
        ProblemDetail pd = ProblemDetail.forStatusAndDetail(HttpStatus.NOT_FOUND, ex.getMessage());
        pd.setTitle("Pedido no encontrado");
        return pd;
    }

    @ExceptionHandler(OutOfStockException.class)
    ProblemDetail outOfStock(OutOfStockException ex) {
        ProblemDetail pd = ProblemDetail.forStatusAndDetail(HttpStatus.CONFLICT, ex.getMessage());
        pd.setTitle("Sin stock");
        pd.setProperty("sku", ex.getSku());
        return pd;
    }

    @ExceptionHandler(InventoryUnavailableException.class)
    ProblemDetail inventoryDown(InventoryUnavailableException ex) {
        return ProblemDetail.forStatusAndDetail(HttpStatus.SERVICE_UNAVAILABLE,
                "El servicio de inventario no está disponible. Inténtalo más tarde.");
    }
}
```

La clase base `ResponseEntityExceptionHandler` ya traduce a `ProblemDetail` las excepciones propias de Spring MVC (validación, métodos no soportados, cuerpo ilegible…).

Actuator, perfiles y buenas costumbres

- **Actuator** expone `health`, `info`, `metrics` y `prometheus`. Publica solo los endpoints necesarios y protégelos.
- **Perfiles** (`application-dev.yml`, `application-prod.yml`) activados con `SPRING_PROFILES_ACTIVE`.
- **Configuración tipada** con `@ConfigurationProperties` y records, en lugar de `@Value` repartido.
- **Inyección por constructor**: facilita las pruebas y hace las dependencias explícitas.
- **Flyway o Liquibase** para versionar el esquema; `ddl-auto=validate` o `none` en producción.
- `open-in-view: false` para evitar sorpresas con transacciones y carga perezosa.

Configuración tipada

```
@ConfigurationProperties(prefix = "clients.inventory")
public record InventoryClientProperties(
        String baseUrl,
        Duration connectTimeout,
        Duration responseTimeout) {}

// application.yml
// clients:
//   inventory:
//     base-url: http://inventory-service
//     connect-timeout: 2s
//     response-timeout: 3s
```

17. Un ejemplo completo: tienda con varios servicios

Usaremos como hilo conductor una pequeña tienda. Muestra los componentes principales y sirve de guía para los capítulos siguientes.

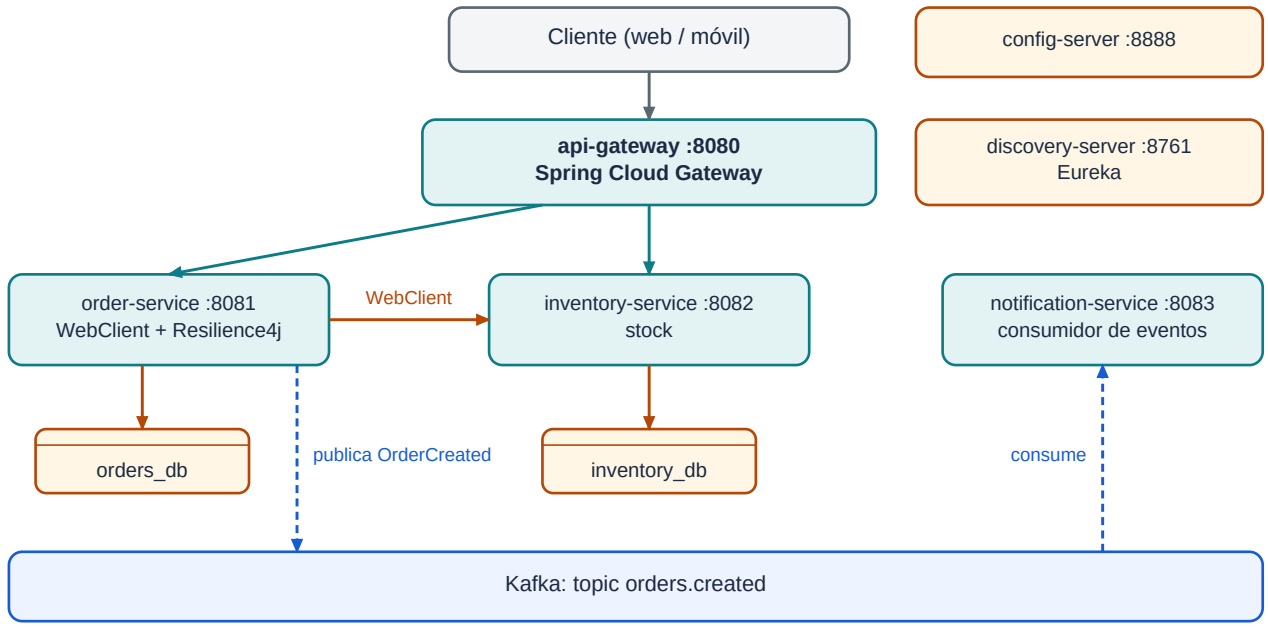


Figura 17.1 · Arquitectura del ejemplo. Línea continua: HTTP. Línea discontinua: eventos.

Servicio	Responsabilidad	Tecnología
api-gateway	Entrada única, enrutado, seguridad, límites, circuit breaker de borde.	Spring Cloud Gateway (WebFlux)
order-service	Crea y consulta pedidos; llama a inventario y publica `OrderCreated`.	Web MVC, JPA, WebClient, Resilience4j, Kafka
inventory-service	Gestiona stock y reservas; expone una API REST.	Web MVC, JPA
notification-service	Consume eventos y envía notificaciones.	Kafka consumer
discovery-server	Registro de instancias.	Eureka Server
config-server	Configuración centralizada.	Spring Cloud Config

Flujo de creación de un pedido: el gateway recibe `POST /api/orders` y lo enruta a `order-service`; este reserva stock llamando por HTTP a `inventory-service` (con timeout, reintento y circuit breaker), guarda el pedido y publica el evento `OrderCreated` para que `notification-service` (y en el futuro pagos o envíos) reaccionen sin acoplarse.

18. Service Discovery con Eureka y balanceo de carga

En un entorno dinámico las instancias aparecen y desaparecen, y sus direcciones cambian. El *service discovery* permite referirse a un servicio por nombre lógico (`inventory-service`) y resolver en tiempo de ejecución a una instancia sana.

Eureka Server

DiscoveryServerApplication.java

```
// dependencia: spring-cloud-starter-netflix-eureka-server
@SpringBootApplication
@EnableEurekaServer
public class DiscoveryServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(DiscoveryServerApplication.class, args);
    }
}
```

application.yml del servidor Eureka

```
server:
  port: 8761
spring:
  application:
    name: discovery-server
eureka:
  client:
    register-with-eureka: false   # el servidor no se registra a sí mismo
    fetch-registry: false
```

Eureka Client

Basta con añadir la dependencia `spring-cloud-starter-netflix-eureka-client` y apuntar al servidor. El servicio se registra solo con el nombre de `spring.application.name`; ya no hace falta `@EnableDiscoveryClient`.

application.yml del cliente

```
eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka/
  instance:
    prefer-ip-address: true
    lease-renewal-interval-in-seconds: 10     # latido
    lease-expiration-duration-in-seconds: 30  # se da por muerta si no hay latido
```

Balanceo del lado del cliente

`Spring Cloud LoadBalancer` elige entre las instancias registradas (por defecto, *round-robin*). Con `WebClient` se activa marcando el *builder* con `@LoadBalanced`, y la URL usa el nombre lógico del servicio.

Cliente con balanceo (lb)

```
@Configuration
public class WebClientConfig {

    @Bean
    @LoadBalanced
    WebClient.Builder loadBalancedWebClientBuilder() {
        return WebClient.builder();
    }

    @Bean
    WebClient inventoryWebClient(@LoadBalanced WebClient.Builder builder) {
        return builder.baseUrl("http://inventory-service").build();
    }
}
```

> **CUIDADO**
> Si necesitas **dos** tipos de `WebClient.Builder` (con y sin balanceo), cualifica los beans. Un `@LoadBalanced` en un builder compartido hace que **todas** las llamadas, incluidas las externas, intenten resolverse por Eureka.

Alternativa: Kubernetes

En Kubernetes, un `Service` llamado `inventory-service` ya resuelve por DNS a un balanceador interno. El cliente usa simplemente `http://inventory-service:8082` y no necesita Eureka ni LoadBalancer. Es la opción recomendada cuando ya despliegas en Kubernetes.

19. Configuración centralizada

Spring Cloud Config

Un *Config Server* sirve la configuración de todos los servicios desde un repositorio Git, con soporte para entornos y versiones. Permite cambiar propiedades sin recompilar y mantener un historial auditable.

ConfigServerApplication.java

```
// dependencia: spring-cloud-config-server
@SpringBootApplication
@EnableConfigServer
public class ConfigServerApplication { ... }
```

application.yml del Config Server

```
server:
  port: 8888
spring:
  cloud:
    config:
      server:
        git:
          uri: https://github.com/acme/config-repo
          default-label: main
          search-paths: '{application}'
```

En el repositorio de configuración se crean ficheros como `order-service.yml` o `order-service-prod.yml`. El cliente importa la configuración al arrancar:

application.yml del cliente

```
# dependencia: spring-cloud-starter-config
spring:
  application:
    name: order-service
  profiles:
    active: dev
  config:
    import: "optional:configserver:http://localhost:8888"
```

Refrescar la configuración en caliente

- Anota los *beans* que dependen de propiedades cambiantes con `@RefreshScope`.
- Expón `refresh` en Actuator y llama a `POST /actuator/refresh` para recargar.
- Para propagar el cambio a todas las instancias, usa **Spring Cloud Bus** (sobre Kafka o RabbitMQ).

Secretos

> **ANTIPATRÓN · SECRETOS EN GIT**
> No guardes contraseñas ni claves en el repositorio de configuración, ni siquiera privado. Usa Vault, el gestor de secretos de tu nube o `Secret` de Kubernetes, y recíbelos como variables de entorno o ficheros montados.

Alternativas modernas

- **Variables de entorno y `ConfigMap` / `Secret` de Kubernetes**, con perfiles de Spring: simple y estándar en la nube.

- **Spring Cloud Kubernetes** para leer ConfigMaps y Secrets directamente y hacer discovery con la API de Kubernetes.
- **Spring Cloud Vault** para integrar secretos de HashiCorp Vault.

20. API Gateway con Spring Cloud Gateway

Spring Cloud Gateway es un gateway reactivo (basado en WebFlux y Netty). Cada petición se compara con un conjunto de **rutas**; cada ruta tiene un destino (`uri`), unos **predicados** (cuándo aplica) y unos **filtros** (qué hacer con la petición o la respuesta).

> **NOTA · NOMBRES SEGÚN VERSIÓN**
> A partir del tren Spring Cloud 2025.0 los starters y las propiedades se han renombrado: la variante reactiva es `spring-cloud-starter-gateway-server-webflux` y las rutas se declaran bajo `spring.cloud.gateway.server.webflux`. En versiones anteriores el starter es `spring-cloud-starter-gateway` y el prefijo es `spring.cloud.gateway`. También existe una variante basada en Servlet (Gateway Server MVC).

Rutas en YAML

Rutas declarativas

```yaml
spring:
  cloud:
    gateway:
      server:
        webflux:
          routes:
            - id: orders
              uri: lb://order-service            # resuelve por discovery
              predicates:
                - Path=/api/orders/**
                - Method=GET,POST
              filters:
                - name: CircuitBreaker
                  args:
                    name: ordersCb
                    fallbackUri: forward:/fallback/orders
                - name: Retry
                  args:
                    retries: 2
                    methods: GET
                    statuses: BAD_GATEWAY, SERVICE_UNAVAILABLE
                    backoff:
                      firstBackoff: 100ms
                      maxBackoff: 1s
                      factor: 2
            - id: inventory
              uri: lb://inventory-service
              predicates:
                - Path=/api/inventory/**
              filters:
                - StripPrefix=1
          default-filters:
            - AddResponseHeader=X-Gateway, spring
```

Rutas en Java

Rutas con la DSL de Java

```java
@Bean
RouteLocator routes(RouteLocatorBuilder builder) {
    return builder.routes()
        .route("orders", r -> r.path("/api/orders/**")
            .filters(f -> f
                .circuitBreaker(c -> c.setName("ordersCb")
                                      .setFallbackUri("forward:/fallback/orders"))
                .retry(cfg -> cfg.setRetries(2).setMethods(HttpMethod.GET)))
```

```
            .uri("lb://order-service"))
        .build();
}
```

Filtros habituales

Filtro	Qué hace
`StripPrefix`, `RewritePath`, `PrefixPath`	Modifican la ruta antes de reenviar.
`AddRequestHeader`, `AddResponseHeader`	Añaden cabeceras.
`CircuitBreaker`	Aplica Resilience4j con *fallback* (requiere `spring-cloud-starter-circuitbreaker-reactor-resilience4j`).
`Retry`	Reintenta peticiones según estado o excepción.
`RequestRateLimiter`	Limita la tasa por clave (usuario, IP) con Redis.
`TokenRelay`	Reenvía el token OAuth2 hacia el servicio.
`RequestSize`, `ModifyRequestBody`	Límites y transformaciones del cuerpo.

Límite de tasa con Redis

Rate limiting

```
# dependencia: spring-boot-starter-data-redis-reactive
- id: orders
  uri: lb://order-service
  predicates: [ "Path=/api/orders/**" ]
  filters:
    - name: RequestRateLimiter
      args:
        redis-rate-limiter.replenishRate: 20    # peticiones/segundo sostenidas
        redis-rate-limiter.burstCapacity: 40    # ráfaga permitida
        key-resolver: "#{@userKeyResolver}"
```

Resolutor de clave

```
@Bean
KeyResolver userKeyResolver() {
    return exchange -> Mono.justOrEmpty(exchange.getRequest().getHeaders().getFirst("X-User-Id"))
                           .defaultIfEmpty("anonymous");
}
```

Filtro global propio: correlación

CorrelationIdFilter.java

```
@Component
public class CorrelationIdFilter implements GlobalFilter, Ordered {
    private static final String HEADER = "X-Correlation-Id";

    @Override
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {
        String id = Optional.ofNullable(exchange.getRequest().getHeaders().getFirst(HEADER))
                            .orElse(UUID.randomUUID().toString());
        ServerHttpRequest request = exchange.getRequest().mutate().header(HEADER, id).build();
        exchange.getResponse().getHeaders().add(HEADER, id);
        return chain.filter(exchange.mutate().request(request).build());
    }

    @Override public int getOrder() { return Ordered.HIGHEST_PRECEDENCE; }
}
```

Buenas prácticas del gateway

- Mantenlo **fino**: enrutado, seguridad, límites y observabilidad. La lógica de negocio no va aquí.
- No lo conviertas en un cuello de botella único: despliega **varias réplicas** detrás de un balanceador.
- Un **BFF** por tipo de cliente (web, móvil) cuando las necesidades de datos difieren.
- Valida el JWT en el gateway **y** en cada servicio (defensa en profundidad).
- Define **timeouts** de respuesta por ruta (`spring.cloud.gateway...httpclient.response-timeout` o metadatos de ruta).
- No mezcles en la misma aplicación el starter del gateway reactivo con `spring-boot-starter-web`.

21. Comunicación síncrona con WebClient

`WebClient` es el cliente HTTP no bloqueante y fluido de Spring. Pertenece a `spring-webflux`, pero se puede usar perfectamente desde una aplicación Spring MVC tradicional. Es la forma recomendada de llamar a otros servicios cuando necesitas **timeouts finos, reintentos, paralelismo o streaming**. Se apoya en Project Reactor: las respuestas son `Mono<T>` (0 o 1 elemento) y `Flux<T>` (0..N elementos).

Elegir un cliente HTTP en Spring

Cliente	Modelo	Cuándo usarlo
`WebClient`	No bloqueante (Reactor). Desde MVC se puede bloquear con `.block()`.	Apps reactivas; llamadas en paralelo; streaming; control fino de reintentos y timeouts.
`RestClient` **(Spring 6.1 / Boot 3.2+)**	Bloqueante, API fluida similar a WebClient.	Apps MVC sencillas; sustituye a `RestTemplate` en proyectos nuevos.
`RestTemplate`	Bloqueante, API clásica.	Código existente. En modo mantenimiento; no para código nuevo.
HTTP Interface (`@HttpExchange`)	Cliente declarativo sobre `WebClient` o `RestClient`.	Contratos limpios tipo Feign, sin dependencias extra de Spring Cloud.
OpenFeign	Declarativo, bloqueante.	Proyectos existentes. Está en modo mantenimiento; para nuevos, prefiere HTTP Interface.

> **IDEA CLAVE**
> Con hilos virtuales (Java 21+) el modelo bloqueante vuelve a escalar muy bien, y `RestClient` es una opción muy cómoda en apps MVC. Elige `WebClient` cuando realmente necesites composición reactiva, paralelismo fino o streaming.

Dependencias

pom.xml

```
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-webflux</artifactId>
  </dependency>
  <!-- tests -->
  <dependency>
    <groupId>io.projectreactor</groupId>
    <artifactId>reactor-test</artifactId>
    <scope>test</scope>
  </dependency>
  <dependency>
    <groupId>com.squareup.okhttp3</groupId>
    <artifactId>mockwebserver</artifactId>
    <scope>test</scope>
  </dependency>
```

Configurar el cliente correctamente

Inyecta siempre el `WebClient.Builder` que ofrece Spring Boot: ya incluye los códecs de Jackson, la propagación del contexto de trazado y las métricas. Crear el cliente con `WebClient.create()` pierde todo eso.

InventoryClientConfig.java

```
@Configuration
@EnableConfigurationProperties(InventoryClientProperties.class)
public class InventoryClientConfig {

    @Bean
    WebClient inventoryWebClient(WebClient.Builder builder, InventoryClientProperties p) {

        ConnectionProvider pool = ConnectionProvider.builder("inventory")
                .maxConnections(100)                          // conexiones máximas al host
                .pendingAcquireTimeout(Duration.ofSeconds(2))// espera máxima por una conexión libre
                .maxIdleTime(Duration.ofSeconds(30))
                .maxLifeTime(Duration.ofMinutes(5))
                .build();

        HttpClient http = HttpClient.create(pool)
                .option(ChannelOption.CONNECT_TIMEOUT_MILLIS, (int) p.connectTimeout().toMillis())
                .responseTimeout(p.responseTimeout());       // tiempo total hasta recibir la respuesta

        return builder
                .baseUrl(p.baseUrl())
                .clientConnector(new ReactorClientHttpConnector(http))
                .defaultHeader(HttpHeaders.ACCEPT, MediaType.APPLICATION_JSON_VALUE)
                .build();
    }
}
```

- **connect timeout:** cuánto esperas a establecer la conexión TCP.
- **response timeout:** cuánto esperas a que llegue la respuesta tras enviar la petición.
- **pool de conexiones:** limita la concurrencia hacia cada dependencia (actúa como *bulkhead* a nivel de conexión).
- El `WebClient` es **inmutable y seguro entre hilos**: crea uno por dependencia y reutilízalo.

Peticiones GET

GET

```
public Mono<StockResponse> getStock(String sku) {
    return client.get()
            .uri("/api/stock/{sku}", sku)           // variables de ruta codificadas de forma segura
            .retrieve()                             // obtiene el cuerpo; error en 4xx/5xx
            .bodyToMono(StockResponse.class);
}

// con parámetros de consulta
client.get()
      .uri(b -> b.path("/api/stock").queryParam("warehouse", "MAD")
                 .queryParam("page", 0).build())
      .retrieve()
      .bodyToFlux(StockResponse.class);             // colección como flujo
```

Peticiones POST con cuerpo

POST

```
public Mono<ReservationResponse> reserve(ReserveRequest request, String idempotencyKey) {
    return client.post()
            .uri("/api/stock/reservations")
            .contentType(MediaType.APPLICATION_JSON)
            .header("Idempotency-Key", idempotencyKey)
            .bodyValue(request)                      // serializa el objeto a JSON
            .retrieve()
            .bodyToMono(ReservationResponse.class);
}

// sin cuerpo en la respuesta
client.delete().uri("/api/stock/reservations/{id}", id)
```

```
       .retrieve().toBodilessEntity();               // Mono<ResponseEntity<Void>>
```

Gestión de errores

`retrieve()` convierte cualquier 4xx o 5xx en una `WebClientResponseException`. Con `onStatus` puedes traducirlas a excepciones de tu dominio, distinguiendo los errores de cliente (no reintentables) de los de servidor (posiblemente transitorios).

Traducción de errores

```
public Mono<StockResponse> getStock(String sku) {
    return client.get().uri("/api/stock/{sku}", sku)
        .retrieve()
        .onStatus(s -> s.value() == 404,
                  r -> Mono.error(new SkuNotFoundException(sku)))
        .onStatus(HttpStatusCode::is4xxClientError,
                  r -> r.bodyToMono(String.class)
                        .defaultIfEmpty("")
                        .map(body -> new InventoryClientException(r.statusCode().value(), body)))
        .onStatus(HttpStatusCode::is5xxServerError,
                  r -> Mono.error(new InventoryUnavailableException(r.statusCode().value())))
        .bodyToMono(StockResponse.class);
}
```

Cuando necesitas inspeccionar la respuesta completa (código, cabeceras y cuerpo), usa `exchangeToMono`. Con `exchange*` **eres responsable de consumir el cuerpo siempre**, incluso si no lo usas, para no perder conexiones del pool.

exchangeToMono

```
client.get().uri("/api/stock/{sku}", sku)
    .exchangeToMono(resp -> {
        if (resp.statusCode().is2xxSuccessful()) {
            return resp.bodyToMono(StockResponse.class);
        } else if (resp.statusCode().value() == 404) {
            return resp.releaseBody().then(Mono.empty());   // libera la conexión
        }
        return resp.createError();                          // WebClientResponseException
    });
```

Timeouts adicionales y reintentos

Además de los timeouts de Netty, puedes limitar el tiempo total de una operación reactiva con `.timeout(Duration)`. Los reintentos se declaran con `Retry.backoff`, que aplica backoff exponencial con *jitter*.

Timeout + retry con backoff

```
public Mono<StockResponse> getStock(String sku) {
    return client.get().uri("/api/stock/{sku}", sku)
        .retrieve()
        .onStatus(HttpStatusCode::is5xxServerError,
                  r -> Mono.error(new InventoryUnavailableException(r.statusCode().value())))
        .bodyToMono(StockResponse.class)
        .timeout(Duration.ofSeconds(3))                      // presupuesto total de la llamada
        .retryWhen(Retry.backoff(3, Duration.ofMillis(200))  // 3 reintentos: 200ms, 400ms, 800ms...
                .maxBackoff(Duration.ofSeconds(2))
                .jitter(0.5)
                .filter(ex -> ex instanceof InventoryUnavailableException
                           || ex instanceof TimeoutException)
                .onRetryExhaustedThrow((spec, signal) -> signal.failure()));
}
```

CUIDADO

Reintenta **solo lecturas o escrituras idempotentes**. Un `POST` que crea un recurso sin clave de idempotencia puede duplicarlo si el primer intento llegó al servidor pero la respuesta se perdió. Sin `onRetryExhaustedThrow`, al agotar los reintentos obtienes una `RetryExhaustedException` que oculta la causa original.

Usar WebClient desde Spring MVC: bloquear o no

En una aplicación MVC (hilo por petición) puedes bloquear el resultado con `.block()`. Es seguro porque ese hilo no pertenece a un *event loop*. Acompáñalo siempre de un timeout.

block() en MVC

```
// En un servicio MVC (bloqueante)
public StockResponse getStockBlocking(String sku) {
    return client.get().uri("/api/stock/{sku}", sku)
            .retrieve()
            .bodyToMono(StockResponse.class)
            .block(Duration.ofSeconds(4));          // nunca .block() sin límite
}
```

ANTIPATRÓN · BLOCK() EN EL SITIO EQUIVOCADO

Nunca llames a `.block()` dentro de un hilo reactivo (un controlador WebFlux, un `flatMap`, un filtro del gateway). Reactor lanza `IllegalStateException: block()/blockFirst()/blockLast() are blocking, which is not supported in thread reactor-http-nio-X`. En un pipeline reactivo, compón con `map`, `flatMap` y `zip` y devuelve el `Mono` hasta el controlador.

Llamadas en paralelo y composición

Una de las grandes ventajas de `WebClient`: lanzar varias llamadas independientes a la vez y combinarlas.

Composición reactiva

```
public Mono<ProductDetail> getDetail(String sku) {
    Mono<StockResponse> stock = inventoryClient.getStock(sku);
    Mono<PriceResponse> price = pricingClient.getPrice(sku);
    Mono<List<Review>> reviews = reviewsClient.getReviews(sku)
            .onErrorReturn(List.of());                  // dependencia opcional: degradar

    return Mono.zip(stock, price, reviews)              // se ejecutan en paralelo
            .map(t -> new ProductDetail(sku, t.getT1(), t.getT2(), t.getT3()));
}

// muchas llamadas con concurrencia limitada
public Flux<StockResponse> getStocks(List<String> skus) {
    return Flux.fromIterable(skus)
               .flatMap(this::getStock, 8);             // máx. 8 llamadas simultáneas
}

// llamadas dependientes (secuenciales)
public Mono<Invoice> invoice(String orderId) {
    return orderClient.get(orderId)
            .flatMap(order -> customerClient.get(order.customerId())
                    .map(customer -> Invoice.of(order, customer)));
}
```

- `Mono.zip` falla si falla cualquiera de las partes: añade `onErrorReturn` / `onErrorResume` para las opcionales.
- En `flatMap`, el segundo argumento limita la concurrencia: protege al servicio llamado.
- Desde MVC puedes combinar con `.block()` **una sola vez al final** del pipeline.

Filtros: autenticación, correlación y logging

Un `ExchangeFilterFunction` intercepta cada petición y respuesta del cliente, como un interceptor.

Filtros del cliente

```
@Bean
WebClient orderWebClient(WebClient.Builder builder) {
    return builder
        .baseUrl("http://order-service")
        .filter(bearerTokenRelay())
        .filter(logErrors())
        .build();
}

// Reenvía el token del usuario autenticado (app MVC con Spring Security)
private ExchangeFilterFunction bearerTokenRelay() {
    return (request, next) -> {
        Authentication auth = SecurityContextHolder.getContext().getAuthentication();
        if (auth instanceof JwtAuthenticationToken jwt) {
            request = ClientRequest.from(request)
                    .headers(h -> h.setBearerAuth(jwt.getToken().getTokenValue()))
                    .build();
        }
        return next.exchange(request);
    };
}

private ExchangeFilterFunction logErrors() {
    return ExchangeFilterFunction.ofResponseProcessor(resp -> {
        if (resp.statusCode().isError()) {
            log.warn("Llamada fallida: {}", resp.statusCode());
        }
        return Mono.just(resp);
    });
}
```

> **NOTA**
> El `SecurityContextHolder` se lee en el hilo que **construye** la petición. Si ejecutas la llamada en otro hilo (por ejemplo, dentro de `Mono.zip` en un pipeline reactivo) el contexto puede no estar disponible. En ese caso, extrae el token antes y pásalo explícitamente, o usa los filtros OAuth2 de Spring Security para WebClient (`ServletOAuth2AuthorizedClientExchangeFilterFunction`, con *client credentials* para llamadas máquina a máquina).

Trazado y correlación: al usar el `WebClient.Builder` de Spring Boot con Micrometer Tracing, las cabeceras de traza (`traceparent`) se propagan automáticamente en las llamadas salientes. Solo necesitas un filtro propio para cabeceras de negocio como `X-Correlation-Id`.

Un cliente de servicio limpio (adaptador de salida)

Encapsula `WebClient` detrás de una clase de infraestructura con un método por operación y un contrato del dominio. El servicio de aplicación no debe conocer `WebClient`, `Mono` ni HTTP.

InventoryClient.java

```
@Component
public class InventoryClient {
    private static final Logger log = LoggerFactory.getLogger(InventoryClient.class);
    private final WebClient client;

    public InventoryClient(@Qualifier("inventoryWebClient") WebClient client) {
        this.client = client;
    }

    public void reserve(String idempotencyKey, List<CreateOrderRequest.Item> items) {
        ReserveRequest body = ReserveRequest.from(items);
```

```
        client.post().uri("/api/stock/reservations")
            .header("Idempotency-Key", idempotencyKey)
            .bodyValue(body)
            .retrieve()
            .onStatus(s -> s.value() == 409, r -> r.bodyToMono(ProblemDetail.class)
                    .map(pd -> new OutOfStockException(String.valueOf(pd.getProperties().get("sku")))))
            .onStatus(HttpStatusCode::is5xxServerError,
                    r -> Mono.error(new InventoryUnavailableException(r.statusCode().value())))
            .toBodilessEntity()
            .timeout(Duration.ofSeconds(3))
            .block(Duration.ofSeconds(4));          // adaptador bloqueante para un servicio MVC
    }
}
```

HTTP Interfaces: clientes declarativos

Desde Spring 6 puedes declarar el cliente como una interfaz y Spring genera la implementación sobre `WebClient`. Es la alternativa moderna y ligera a OpenFeign.

HTTP Interface

```
@HttpExchange(url = "/api/stock", accept = "application/json")
public interface InventoryApi {

    @GetExchange("/{sku}")
    Mono<StockResponse> getStock(@PathVariable String sku);

    @PostExchange("/reservations")
    Mono<ReservationResponse> reserve(@RequestBody ReserveRequest request,
                                      @RequestHeader("Idempotency-Key") String key);
}

@Bean
InventoryApi inventoryApi(@Qualifier("inventoryWebClient") WebClient webClient) {
    HttpServiceProxyFactory factory = HttpServiceProxyFactory
            .builderFor(WebClientAdapter.create(webClient)).build();
    return factory.createClient(InventoryApi.class);
}
```

En versiones recientes de Spring Framework (7) y Spring Boot (4) el registro de estas interfaces se simplifica con `@ImportHttpServices` y propiedades de configuración. Consulta la documentación de tu versión.

RestClient: el equivalente bloqueante

RestClient (Spring 6.1+)

```
RestClient client = restClientBuilder.baseUrl("http://inventory-service").build();

StockResponse stock = client.get()
        .uri("/api/stock/{sku}", sku)
        .retrieve()
        .onStatus(HttpStatusCode::is5xxServerError, (req, res) -> {
            throw new InventoryUnavailableException(res.getStatusCode().value());
        })
        .body(StockResponse.class);
```

Probar un cliente WebClient

MockWebServer + StepVerifier

```
class InventoryClientTest {
    private MockWebServer server;
    private InventoryClientReactive client;

    @BeforeEach void setUp() throws IOException {
        server = new MockWebServer();
        server.start();
        WebClient webClient = WebClient.builder().baseUrl(server.url("/").toString()).build();
        client = new InventoryClientReactive(webClient);
```

```
    }
    @AfterEach void tearDown() throws IOException { server.shutdown(); }

    @Test void devuelveElStock() {
        server.enqueue(new MockResponse()
                .setHeader("Content-Type", "application/json")
                .setBody("{\"sku\":\"A1\",\"available\":5}"));

        StepVerifier.create(client.getStock("A1"))
                .expectNextMatches(r -> r.available() == 5)
                .verifyComplete();
    }

    @Test void reintentaTrasUn503() {
        server.enqueue(new MockResponse().setResponseCode(503));
        server.enqueue(new MockResponse().setHeader("Content-Type", "application/json")
                .setBody("{\"sku\":\"A1\",\"available\":5}"));

        StepVerifier.create(client.getStock("A1"))
                .expectNextCount(1).verifyComplete();
        assertThat(server.getRequestCount()).isEqualTo(2);
    }
}
```

Alternativas: **WireMock** (simula latencias y fallos de forma muy expresiva) y **Testcontainers** con el servicio real.

Errores frecuentes con WebClient

Error	Consecuencia	Solución
`.block()` **en un hilo de Netty**	Excepción o bloqueo del event loop.	Compón reactivo o ejecuta en un hilo adecuado.
Sin timeouts	Hilos y conexiones agotadas ante un servicio lento.	Connect, response y `.timeout()` siempre.
`WebClient.create()` **en cada llamada**	Se pierden el pool, el tracing y la configuración de Boot.	Un bean por dependencia, creado desde el `Builder`.
Reintentar `POST` **no idempotente**	Duplicados.	Claves de idempotencia o no reintentar.
`exchange*` **sin consumir el cuerpo**	Fuga de conexiones.	Consumir o `releaseBody()`; preferir `retrieve()`.
Ignorar el error en un `Mono` **que nadie suscribe**	La llamada nunca se ejecuta.	Un `Mono` es perezoso: hay que devolverlo o suscribirse (`block`/`subscribe`).
Cargar respuestas enormes en memoria	`DataBufferLimitException` (límite de 256 KB por defecto).	Streaming con `Flux<DataBuffer>` o subir `maxInMemorySize` con criterio.
Un `@LoadBalanced` **global**	Las URLs externas se intentan resolver por discovery.	Builders diferenciados con `@Qualifier`.

22. Resiliencia con Resilience4j

Resilience4j es la librería de referencia en Spring para circuit breaker, retry, bulkhead, rate limiter y time limiter. Es ligera, modular y se integra con Spring Boot, Actuator y Micrometer.

Dependencias

pom.xml

```
<dependency>
  <groupId>io.github.resilience4j</groupId>
  <artifactId>resilience4j-spring-boot3</artifactId>
</dependency>
<dependency>
  <groupId>io.github.resilience4j</groupId>
  <artifactId>resilience4j-reactor</artifactId>    <!-- soporte para Mono / Flux -->
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-aop</artifactId> <!-- necesario para las anotaciones -->
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

Como alternativa de más alto nivel existe **Spring Cloud CircuitBreaker** (`spring-cloud-starter-circuitbreaker-resilience4j` para MVC y `...-reactor-resilience4j` para reactivo), que abstrae la implementación y es la que usa el filtro `CircuitBreaker` del Gateway.

Configuración en YAML

application.yml

```
resilience4j:
  circuitbreaker:
    instances:
      inventory:
        sliding-window-type: COUNT_BASED
        sliding-window-size: 20                   # últimas 20 llamadas
        minimum-number-of-calls: 10               # no evalúa hasta tener 10
        failure-rate-threshold: 50                # abre al 50 % de fallos
        slow-call-duration-threshold: 2s          # una llamada > 2s cuenta como lenta
        slow-call-rate-threshold: 80
        wait-duration-in-open-state: 15s          # tiempo en ABIERTO
        permitted-number-of-calls-in-half-open-state: 5
        automatic-transition-from-open-to-half-open-enabled: true
        record-exceptions:
          - com.acme.orders.InventoryUnavailableException
          - java.util.concurrent.TimeoutException
        ignore-exceptions:
          - com.acme.orders.OutOfStockException    # error de negocio: no es un fallo del servicio
  retry:
    instances:
      inventory:
        max-attempts: 3
        wait-duration: 200ms
        enable-exponential-backoff: true
        exponential-backoff-multiplier: 2
        retry-exceptions:
          - com.acme.orders.InventoryUnavailableException
  bulkhead:
    instances:
      inventory:
        max-concurrent-calls: 20
        max-wait-duration: 0
```

```
  ratelimiter:
    instances:
      inventory:
        limit-for-period: 50
        limit-refresh-period: 1s
        timeout-duration: 0
  timelimiter:
    instances:
      inventory:
        timeout-duration: 3s
```

Con anotaciones

Anotaciones sobre un componente

```
@Component
public class InventoryGateway {
    private final InventoryClient client;
    public InventoryGateway(InventoryClient client) { this.client = client; }

    @CircuitBreaker(name = "inventory", fallbackMethod = "reserveFallback")
    @Retry(name = "inventory")
    @Bulkhead(name = "inventory")
    public void reserve(String key, List<CreateOrderRequest.Item> items) {
        client.reserve(key, items);
    }

    // mismos parámetros + la excepción; el tipo de retorno debe coincidir
    private void reserveFallback(String key, List<CreateOrderRequest.Item> items, Throwable ex) {
        if (ex instanceof OutOfStockException) throw (OutOfStockException) ex;
        throw new InventoryUnavailableException(503);   // error limpio y controlado
    }
}
```

> **NOTA**
> Orden de los aspectos por defecto (de fuera hacia dentro): **Retry** → **CircuitBreaker** → **RateLimiter** → **TimeLimiter** → **Bulkhead** → **llamada**. Es decir, cada reintento pasa por el circuit breaker. Las anotaciones funcionan mediante proxies de AOP: no se aplican si llamas al método desde la misma clase (*self-invocation*).

Con operadores reactivos

Si el cliente devuelve `Mono`/`Flux`, aplica los operadores directamente, sin AOP:

Operadores de Resilience4j con Reactor

```
public Mono<StockResponse> getStock(String sku) {
    return client.getStock(sku)
            .transformDeferred(CircuitBreakerOperator.of(
                    circuitBreakerRegistry.circuitBreaker("inventory")))
            .transformDeferred(RetryOperator.of(retryRegistry.retry("inventory")))
            .transformDeferred(BulkheadOperator.of(bulkheadRegistry.bulkhead("inventory")))
            .onErrorResume(CallNotPermittedException.class,         // circuito abierto
                           ex -> Mono.just(StockResponse.unknown(sku)));
}
```

Qué debe considerarse fallo

- **Sí:** timeouts, errores de conexión, 502/503/504, excepciones de infraestructura.
- **No:** errores de validación (400), recurso inexistente (404), reglas de negocio (409). Configúralo con `ignore-exceptions` o `record-exceptions`.
- Si todos los errores abren el circuito, un usuario que envía datos inválidos podría tirar el servicio para todos.

Observar los cortacircuitos

Actuator

```
management:
  endpoints.web.exposure.include: health,metrics,prometheus,circuitbreakers,retries
  health:
    circuitbreakers.enabled: true
# Métricas en Prometheus: resilience4j_circuitbreaker_state,
# resilience4j_circuitbreaker_calls_seconds_count, resilience4j_retry_calls_total ...
```

Pautas de dimensionamiento

Parámetro	Cómo elegirlo
Timeout	Algo por encima del p99 de la dependencia; siempre menor que el del llamante.
Reintentos	2 o 3 como máximo, con backoff exponencial y jitter.
Umbral de fallos	40-60 %, con un mínimo de llamadas suficiente (≥ 10) para evitar falsos positivos.
Tiempo en ABIERTO	Lo que tarda razonablemente en recuperarse la dependencia (10-60 s).
Bulkhead	Concurrencia que la dependencia aguanta, con margen; suele coincidir con el tamaño del pool de conexiones.

23. Mensajería asíncrona con Kafka

La comunicación por eventos desacopla a los servicios en el tiempo. Con Spring hay dos caminos principales: **Spring for Apache Kafka** (control fino, API específica de Kafka) y **Spring Cloud Stream** (modelo funcional agnóstico del broker).

Spring for Apache Kafka

application.yml

```
# dependencia: org.springframework.kafka:spring-kafka
spring:
  kafka:
    bootstrap-servers: localhost:9092
    producer:
      key-serializer: org.apache.kafka.common.serialization.StringSerializer
      value-serializer: org.springframework.kafka.support.serializer.JsonSerializer
      acks: all                                  # espera confirmación de todas las réplicas
      properties:
        enable.idempotence: true
        spring.json.add.type.headers: false
    consumer:
      group-id: notification-service
      auto-offset-reset: earliest
      enable-auto-commit: false
      key-deserializer: org.apache.kafka.common.serialization.StringDeserializer
      value-deserializer: org.springframework.kafka.support.serializer.JsonDeserializer
      properties:
        spring.json.trusted.packages: com.acme.events
        spring.json.value.default.type: com.acme.events.OrderCreated
    listener:
      ack-mode: record
```

Productor

```
public record OrderCreated(UUID eventId, UUID orderId, String customerId,
                           BigDecimal total, Instant occurredAt) {}

@Component
public class OrderEventPublisher {
    private final KafkaTemplate<String, Object> kafka;
    public OrderEventPublisher(KafkaTemplate<String, Object> kafka) { this.kafka = kafka; }

    public void orderCreated(Order order) {
        OrderCreated event = new OrderCreated(UUID.randomUUID(), order.getId(),
                order.getCustomerId(), order.getTotal(), Instant.now());
        kafka.send("orders.created", order.getId().toString(), event)   // la clave fija la partición
             .whenComplete((result, ex) -> {
                 if (ex != null) log.error("No se pudo publicar {}", event.eventId(), ex);
             });
    }
}
```

Consumidor idempotente

```
@Component
public class OrderCreatedListener {

    @KafkaListener(topics = "orders.created", groupId = "notification-service")
    public void on(OrderCreated event) {
        // idempotencia: ¿ya procesé este eventId?
        if (processedEvents.exists(event.eventId())) return;
        notifications.sendOrderConfirmation(event);
        processedEvents.save(event.eventId());
    }
}
```

Gestión de errores en consumidores

- **Reintentos con backoff** (`DefaultErrorHandler` con `ExponentialBackOff`) para fallos transitorios.
- **Dead Letter Topic** (`DeadLetterPublishingRecoverer`) para mensajes que siguen fallando; así no bloquean la partición.
- **Excepciones no reintentables** (mensaje mal formado): al DLT directamente.
- **Monitoriza el lag** del grupo de consumidores y el tamaño del DLT.

Reintentos y Dead Letter Topic

```
@Bean
DefaultErrorHandler errorHandler(KafkaTemplate<Object, Object> template) {
    var recoverer = new DeadLetterPublishingRecoverer(template);          // -> orders.created.DLT
    var backoff = new ExponentialBackOffWithMaxRetries(4);
    backoff.setInitialInterval(500);
    backoff.setMultiplier(2.0);
    var handler = new DefaultErrorHandler(recoverer, backoff);
    handler.addNotRetryableExceptions(IllegalArgumentException.class);
    return handler;
}
```

Spring Cloud Stream (modelo funcional)

Consumidor con Spring Cloud Stream

```
@Bean
public Consumer<OrderCreated> orderCreated() {
    return event -> notifications.sendOrderConfirmation(event);
}

# application.yml
spring:
  cloud:
    function:
      definition: orderCreated
    stream:
      bindings:
        orderCreated-in-0:
          destination: orders.created
          group: notification-service
```

El mismo código funciona sobre Kafka o RabbitMQ cambiando el *binder*. Es una buena opción cuando quieres independizarte del broker.

Garantías de entrega y orden

- Kafka garantiza el **orden dentro de una partición**: usa como clave el identificador de la entidad (`orderId`) para que sus eventos se ordenen.
- El paralelismo máximo de un grupo es el **número de particiones**.
- Con `acks=all` + `enable.idempotence=true` el productor evita duplicados por reintentos.
- La entrega al consumidor es **al menos una vez**: diseña consumidores idempotentes.
- Versiona los eventos (campo `version` o Schema Registry con Avro/Protobuf) y mantén compatibilidad.

> **BUENA PRÁCTICA**
> Nombra los eventos en pasado (`OrderCreated`, no `CreateOrder`) y mantenlos pequeños y estables: incluyen lo necesario para que el consumidor actúe, sin exponer el modelo interno del emisor.

24. Datos, transacciones y sagas con Spring

Persistencia por servicio

- Cada servicio con su propio `DataSource`, esquema y credenciales.
- Migraciones con **Flyway** (`db/migration/V1__...sql`), ejecutadas al arrancar.
- Estrategia **expand/contract** para cambios compatibles: añade la nueva columna, migra, y solo después elimina la antigua.
- Transacciones locales con `@Transactional` en la capa de aplicación; transacciones **cortas** y sin llamadas de red dentro cuando sea posible.
- **Bloqueo optimista** (`@Version`) para evitar actualizaciones perdidas entre instancias.

> **CUIDADO**
> Llamar a otro servicio por HTTP dentro de un método `@Transactional` mantiene abierta una transacción (y una conexión del pool) durante toda la llamada. Si la dependencia es lenta, agotas el pool. Reserva primero, o confirma la transacción antes de las llamadas largas, y usa la saga para compensar.

Outbox transaccional con Spring

Guardar pedido y evento a la vez

```
@Entity @Table(name = "outbox_event")
public class OutboxEvent {
    @Id private UUID id = UUID.randomUUID();
    private String aggregateId;
    private String topic;
    @Column(columnDefinition = "jsonb") private String payload;
    private Instant createdAt = Instant.now();
    private Instant publishedAt;               // null = pendiente
    // getters/setters
}

@Service
public class OrderService {
    @Transactional
    public OrderResponse placeOrder(CreateOrderRequest req) {
        Order order = orderRepository.save(new Order(req.customerId(), total(req)));
        outboxRepository.save(OutboxEvent.of("orders.created", order.getId(),
                json.write(OrderCreated.from(order))));   // misma transacción que el pedido
        return OrderResponse.from(order);
    }
}
```

Relay programado

```
@Component
public class OutboxRelay {
    private final OutboxRepository outbox;
    private final KafkaTemplate<String, String> kafka;
    // constructor...

    @Scheduled(fixedDelay = 500)
    @Transactional
    public void publishPending() {
        // SELECT ... FOR UPDATE SKIP LOCKED permite varias instancias sin duplicar trabajo
        List<OutboxEvent> batch = outbox.lockNextBatch(100);
        for (OutboxEvent e : batch) {
            kafka.send(e.getTopic(), e.getAggregateId(), e.getPayload()).join();
            e.markPublished();
        }
```

```
    }
}
```

Para volúmenes altos o latencia mínima, sustituye el *poller* por **CDC con Debezium**, que lee el log de transacciones de la base de datos y publica los cambios de la tabla `outbox` en Kafka sin consultas periódicas.

Saga coreografiada

Cada servicio reacciona a los eventos de los demás. Ejemplo: pedido → stock → pago.

Pasos de la saga (resumen)

```
// order-service: arranca la saga y escucha el resultado
@KafkaListener(topics = "payments.completed")
@Transactional
public void onPaymentCompleted(PaymentCompleted e) {
    orderRepository.findById(e.orderId()).ifPresent(Order::confirm);
}

@KafkaListener(topics = {"stock.rejected", "payments.failed"})
@Transactional
public void onFailure(SagaFailed e) {
    orderRepository.findById(e.orderId()).ifPresent(Order::cancel);   // compensación
}

// inventory-service
@KafkaListener(topics = "orders.created")
@Transactional
public void onOrderCreated(OrderCreated e) {
    if (stock.tryReserve(e.orderId(), e.items())) publish("stock.reserved", e);
    else publish("stock.rejected", e);
}

// payment-service: escucha stock.reserved, cobra y publica payments.completed / payments.failed
// y, si el cobro falla, inventory-service escucha payments.failed y libera la reserva
```

Saga orquestada

Un componente (el orquestador) mantiene el estado de la saga y decide el siguiente paso. Puede ser un servicio propio con una máquina de estados persistida o un motor como **Temporal** o **Camunda**. Es preferible cuando el flujo tiene muchos pasos, ramas o necesita visibilidad centralizada.

Orquestador (esquema)

```
public enum SagaState { STARTED, STOCK_RESERVED, COMPLETED, COMPENSATING, FAILED }

@Service
public class OrderSagaOrchestrator {
    public void start(UUID id)           { go(id, STARTED);        send("ReserveStock", id); }
    public void onStockReserved(UUID id) { go(id, STOCK_RESERVED); send("ChargePayment", id); }
    public void onPaymentDone(UUID id)   { go(id, COMPLETED);      send("ConfirmOrder", id); }
    public void onPaymentFailed(UUID id) { go(id, COMPENSATING);   send("ReleaseStock", id); }
}
```

Consumidor idempotente: tabla de mensajes procesados

Idempotencia en el consumidor

```
@Transactional
public void handle(OrderCreated e) {
    // la clave primaria (event_id) impide procesar dos veces el mismo mensaje
    if (!processedEvents.tryInsert(e.eventId())) return;
    reserveStock(e);
}
// Tabla: processed_event(event_id UUID PRIMARY KEY, processed_at TIMESTAMP)
```

25. Observabilidad con Spring Boot

Dependencias y configuración

pom.xml

```
  <dependency><groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId></dependency>
  <dependency><groupId>io.micrometer</groupId>
    <artifactId>micrometer-registry-prometheus</artifactId></dependency>
  <dependency><groupId>io.micrometer</groupId>
    <artifactId>micrometer-tracing-bridge-otel</artifactId></dependency>
  <dependency><groupId>io.opentelemetry</groupId>
    <artifactId>opentelemetry-exporter-otlp</artifactId></dependency>
```

application.yml

```
  management:
    endpoints:
      web:
        exposure:
          include: health,info,prometheus,metrics
    endpoint:
      health:
        probes.enabled: true
        show-details: when-authorized
        group:
          readiness.include: readinessState,db
    tracing:
      sampling.probability: 0.1          # 10 %; 1.0 solo en desarrollo
    otlp:
      tracing.endpoint: http://otel-collector:4318/v1/traces
    metrics:
      tags:
        application: ${spring.application.name}
      distribution:
        percentiles-histogram:
          http.server.requests: true      # histogramas para calcular p95/p99 en Prometheus

  logging:
    pattern:
      level: "%5p [${spring.application.name:},%X{traceId:-},%X{spanId:-}]"
```

> **NOTA**
> En Spring Boot 4 el arranque de OpenTelemetry se simplifica con un starter propio; revisa la documentación de tu versión. La idea es la misma: Micrometer Observation genera métricas y trazas, y OpenTelemetry las exporta por OTLP.

Qué se instrumenta automáticamente

- **Peticiones HTTP entrantes** (`http.server.requests`) con método, ruta, estado y duración.
- **Llamadas HTTP salientes** con `WebClient.Builder`, `RestClient.Builder` o `RestTemplateBuilder` de Boot.
- **JDBC, Kafka, Redis** y otros, según las librerías presentes.
- **JVM, hilos, GC, pool de Hikari, Tomcat/Netty**, listos para Prometheus.
- El `traceId` y el `spanId` aparecen en el MDC y, por tanto, en cada línea de log.

Métricas y observaciones propias

Métrica de negocio y observación

```
@Service
public class OrderService {
    private final Counter ordersCreated;
    private final ObservationRegistry registry;

    public OrderService(MeterRegistry meters, ObservationRegistry registry) {
        this.ordersCreated = Counter.builder("orders.created").tag("channel", "web").register(meters);
        this.registry = registry;
    }

    public OrderResponse placeOrder(CreateOrderRequest req) {
        return Observation.createNotStarted("order.place", registry)
                .lowCardinalityKeyValue("channel", "web")
                .observe(() -> {
                    OrderResponse r = doPlaceOrder(req);
                    ordersCreated.increment();
                    return r;
                });
    }
}
```

> **CUIDADO**
> Las etiquetas (tags) de métricas deben tener **baja cardinalidad**. Un `userId` o un `orderId` como etiqueta crea millones de series y tumba Prometheus. Para identificadores usa logs y trazas.

Health checks y Kubernetes

- `/actuator/health/liveness`: ¿está vivo el proceso? Mal configurado provoca reinicios innecesarios.
- `/actuator/health/readiness`: ¿puede recibir tráfico? Incluye dependencias imprescindibles (por ejemplo, la base de datos), no las opcionales.
- No incluyas en liveness dependencias externas: si cae la base de datos reiniciarías todos los pods a la vez.

Pila de observabilidad recomendada

Necesidad	Opción de código abierto
Métricas	Prometheus + Grafana
Logs	Loki (con Grafana) o Elastic/OpenSearch
Trazas	Tempo o Jaeger, alimentados por un OpenTelemetry Collector
Alertas	Alertmanager / Grafana Alerting
Panel unificado	Grafana, con enlaces entre métricas, logs y trazas por `traceId`

26. Seguridad con Spring Security y OAuth2

Patrón general

1. El usuario se autentica contra un **proveedor de identidad** (Keycloak, Auth0, Entra ID) y obtiene un **JWT**.
2. El cliente envía el JWT al gateway (`Authorization: Bearer ...`).
3. El gateway valida el token y lo reenvía al servicio (**token relay**).
4. Cada servicio actúa como **Resource Server**: valida firma, emisor y caducidad, y autoriza según scopes y roles.
5. Entre servicios sin usuario (procesos internos) se usa **client credentials**.

Servicio como Resource Server

application.yml

```yaml
# dependencia: spring-boot-starter-oauth2-resource-server
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri: http://keycloak:8080/realms/shop     # descarga las claves públicas (JWKS)
```

SecurityConfig.java

```java
@Configuration
@EnableMethodSecurity
public class SecurityConfig {

    @Bean
    SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        return http
            .csrf(csrf -> csrf.disable())                          // API sin sesión ni cookies
            .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/actuator/health/**").permitAll()
                .requestMatchers(HttpMethod.GET, "/api/orders/**").hasAuthority("SCOPE_orders:read")
                .requestMatchers(HttpMethod.POST, "/api/orders/**").hasAuthority("SCOPE_orders:write")
                .anyRequest().authenticated())
            .oauth2ResourceServer(oauth -> oauth.jwt(Customizer.withDefaults()))
            .build();
    }
}
```

Autorización fina y roles

```java
@PreAuthorize("#customerId == authentication.token.subject or hasRole('ADMIN')")
public Page<OrderResponse> listByCustomer(String customerId, Pageable pageable) { ... }

// Roles de Keycloak: convertirlos a authorities con un JwtAuthenticationConverter
@Bean
JwtAuthenticationConverter jwtAuthenticationConverter() {
    JwtGrantedAuthoritiesConverter scopes = new JwtGrantedAuthoritiesConverter();
    JwtAuthenticationConverter converter = new JwtAuthenticationConverter();
    converter.setJwtGrantedAuthoritiesConverter(jwt -> {
        Collection<GrantedAuthority> auth = new ArrayList<>(scopes.convert(jwt));
        Map<String, Object> realm = jwt.getClaimAsMap("realm_access");
        if (realm != null && realm.get("roles") instanceof Collection<?> roles) {
            roles.forEach(r -> auth.add(new SimpleGrantedAuthority("ROLE_" + r)));
        }
        return auth;
```

```
        });
        return converter;
    }
```

Gateway: token relay

Con `spring-cloud-starter-gateway-server-webflux` y el cliente OAuth2, el filtro `TokenRelay` reenvía el token del usuario a los servicios. Si el gateway solo valida el JWT, basta con que reenvíe la cabecera `Authorization`, que lo hace por defecto.

Llamadas entre servicios

- **En nombre del usuario:** reenvía su JWT (filtro de `WebClient` del capítulo 21).
- **Como servicio (client credentials):** `ServletOAuth2AuthorizedClientExchangeFilterFunction` con un `OAuth2AuthorizedClientManager` y un registro de cliente en `spring.security.oauth2.client.registration`.
- Valida la **audiencia** (`aud`) del token para que un token emitido para otro servicio no sea aceptado.

Lista de verificación de seguridad en Spring

- Endpoints de Actuator limitados: solo `health` y `prometheus` accesibles y, si es posible, en un puerto de gestión separado (`management.server.port`).
- Dependencias actualizadas (Dependabot/Renovate) y análisis de vulnerabilidades en CI.
- Validación de entrada con Bean Validation en todos los endpoints; límites de tamaño de cuerpo.
- CORS configurado con orígenes explícitos, no con `*` para APIs con credenciales.
- Sin secretos en `application.yml`: variables de entorno o gestor de secretos.
- Logs sin tokens, contraseñas ni datos personales en claro.

27. Testing de microservicios con Spring Boot

Pruebas por capas

Anotación / herramienta	Qué carga	Uso
JUnit 5 + Mockito	Nada de Spring.	Lógica de dominio y servicios con dependencias simuladas.
`@WebMvcTest`	Solo la capa web (controladores, validación, errores).	Contrato HTTP y manejo de errores.
`@DataJpaTest`	Solo JPA y repositorios.	Consultas, mapeos y migraciones con BD real (Testcontainers).
`@SpringBootTest`	El contexto completo.	Integración de extremo a extremo del servicio.
Testcontainers	PostgreSQL, Kafka, Redis, etc., reales en Docker.	Entorno fiable sin mocks de infraestructura.
WireMock / MockWebServer	Servidor HTTP simulado.	Probar clientes `WebClient`, timeouts y fallos.
Spring Cloud Contract / Pact	Contratos consumidor-proveedor.	Evitar roturas entre servicios.

Test de controlador

`OrderControllerTest.java`

```
@WebMvcTest(OrderController.class)
class OrderControllerTest {

    @Autowired MockMvc mvc;
    @MockitoBean OrderService service;      // Boot 3.4+; antes @MockBean

    @Test
    void creaUnPedido() throws Exception {
        var response = new OrderResponse(UUID.randomUUID(), OrderStatus.PENDING,
                                         new BigDecimal("30.00"));
        when(service.placeOrder(eq("key-1"), any())).thenReturn(response);

        mvc.perform(post("/api/orders")
                .header("Idempotency-Key", "key-1")
                .contentType(MediaType.APPLICATION_JSON)
                .content("""
                    {"customerId":"c1","items":[{"sku":"A1","quantity":2,"unitPrice":15.00}]}
                    """))
            .andExpect(status().isCreated())
            .andExpect(jsonPath("$.status").value("PENDING"));
    }

    @Test
    void rechazaUnPedidoSinLineas() throws Exception {
        mvc.perform(post("/api/orders")
                .header("Idempotency-Key", "k").contentType(MediaType.APPLICATION_JSON)
                .content("{\"customerId\":\"c1\",\"items\":[]}"))
            .andExpect(status().isBadRequest());
    }
}
```

Test de repositorio con Testcontainers

OrderRepositoryTest.java

```
@Testcontainers
@DataJpaTest
@AutoConfigureTestDatabase(replace = AutoConfigureTestDatabase.Replace.NONE)
class OrderRepositoryTest {

    @Container
    @ServiceConnection                          // Boot 3.1+: configura el DataSource solo
    static PostgreSQLContainer<?> postgres = new PostgreSQLContainer<>("postgres:16-alpine");

    @Autowired OrderRepository repository;

    @Test
    void guardaYBuscaPorCliente() {
        repository.save(new Order("c1", new BigDecimal("10")));
        var page = repository.findByCustomerId("c1", PageRequest.of(0, 10));
        assertThat(page.getTotalElements()).isEqualTo(1);
    }
}
```

Test de integración con servicios simulados

Prueba de resiliencia con WireMock

```
@SpringBootTest(webEnvironment = RANDOM_PORT)
@Testcontainers
class PlaceOrderIT {
    @Container @ServiceConnection
    static PostgreSQLContainer<?> db = new PostgreSQLContainer<>("postgres:16-alpine");
    @RegisterExtension static WireMockExtension inventory = WireMockExtension.newInstance()
            .options(wireMockConfig().dynamicPort()).build();

    @DynamicPropertySource
    static void props(DynamicPropertyRegistry r) {
        r.add("clients.inventory.base-url", inventory::baseUrl);
    }

    @Test
    void siInventarioTardaDemasiadoDevuelve503(@Autowired TestRestTemplate rest) {
        inventory.stubFor(post(urlPathEqualTo("/api/stock/reservations"))
                .willReturn(ok().withFixedDelay(5_000)));          // más que el timeout configurado

        var response = rest.postForEntity("/api/orders", request(), ProblemDetail.class);

        assertThat(response.getStatusCode()).isEqualTo(HttpStatus.SERVICE_UNAVAILABLE);
    }
}
```

Contratos

- **Spring Cloud Contract:** el proveedor define contratos (Groovy/YAML) que generan tests y *stubs* que el consumidor usa en sus pruebas.
- **Pact:** el consumidor genera el contrato en sus tests; el proveedor lo verifica en su CI. Soporta HTTP y mensajería.
- Publica los contratos en un *broker* (Pact Broker, repositorio de artefactos) y falla la CI del proveedor si rompe a un consumidor.

Consejos

- Mantén las pruebas **rápidas**: el contexto de Spring se cachea entre tests con la misma configuración, evita `@DirtiesContext`.

- Prueba los **caminos de fallo** tanto como el camino feliz: timeouts, 5xx, circuito abierto, mensajes duplicados.
- Reutiliza los contenedores entre tests (`static` y, si procede, *singleton containers*).
- Usa **ArchUnit** para proteger la arquitectura (el dominio no depende de infraestructura).

28. Contenedores y despliegue de servicios Spring Boot

Imagen con Dockerfile

Dockerfile multietapa

```
# ---- build ----
FROM eclipse-temurin:21-jdk AS build
WORKDIR /workspace
COPY pom.xml mvnw ./
COPY .mvn .mvn
RUN ./mvnw -q dependency:go-offline
COPY src src
RUN ./mvnw -q -DskipTests package

# ---- runtime ----
FROM eclipse-temurin:21-jre
RUN groupadd -r app && useradd -r -g app app
WORKDIR /app
COPY --from=build /workspace/target/*.jar app.jar
USER app
EXPOSE 8081
ENV JAVA_TOOL_OPTIONS="-XX:MaxRAMPercentage=75 -XX:+ExitOnOutOfMemoryError"
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Alternativa sin Dockerfile: `./mvnw spring-boot:build-image` genera una imagen optimizada con **Cloud Native Buildpacks**, con capas bien separadas, un usuario no root y el *memory calculator* de la JVM configurado.

Docker Compose para desarrollo local

docker-compose.yml (fragmento)

```
services:
  postgres:
    image: postgres:16-alpine
    environment: { POSTGRES_DB: orders, POSTGRES_USER: orders, POSTGRES_PASSWORD: orders }
    ports: ["5432:5432"]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U orders"]
      interval: 5s
      retries: 10

  order-service:
    build: ./order-service
    environment:
      DB_USER: orders
      DB_PASSWORD: orders
      SPRING_DATASOURCE_URL: jdbc:postgresql://postgres:5432/orders
      CLIENTS_INVENTORY_BASE_URL: http://inventory-service:8082
    depends_on:
      postgres: { condition: service_healthy }
    ports: ["8081:8081"]
```

Spring Boot 3.1+ incluye soporte nativo para Docker Compose y Testcontainers en desarrollo (`spring-boot-docker-compose`, `@ServiceConnection`): al arrancar la aplicación se levantan las dependencias solas.

Despliegue en Kubernetes

Deployment, Service y HPA

```
apiVersion: apps/v1
kind: Deployment
metadata: { name: order-service }
spec:
  replicas: 3
  selector: { matchLabels: { app: order-service } }
  strategy:
    rollingUpdate: { maxUnavailable: 0, maxSurge: 1 }
  template:
    metadata: { labels: { app: order-service } }
    spec:
      terminationGracePeriodSeconds: 40
      containers:
        - name: app
          image: registry.acme.io/order-service:1.4.2
          ports: [ { containerPort: 8081 } ]
          env:
            - name: SPRING_PROFILES_ACTIVE
              value: prod
            - name: DB_PASSWORD
              valueFrom: { secretKeyRef: { name: orders-db, key: password } }
          resources:
            requests: { cpu: "250m", memory: "512Mi" }
            limits:   { memory: "768Mi" }
          startupProbe:
            httpGet: { path: /actuator/health/liveness, port: 8081 }
            failureThreshold: 30
            periodSeconds: 2
          livenessProbe:
            httpGet: { path: /actuator/health/liveness, port: 8081 }
            periodSeconds: 10
          readinessProbe:
            httpGet: { path: /actuator/health/readiness, port: 8081 }
            periodSeconds: 5
---
apiVersion: v1
kind: Service
metadata: { name: order-service }
spec:
  selector: { app: order-service }
  ports: [ { port: 80, targetPort: 8081 } ]
---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata: { name: order-service }
spec:
  scaleTargetRef: { apiVersion: apps/v1, kind: Deployment, name: order-service }
  minReplicas: 3
  maxReplicas: 12
  metrics:
    - type: Resource
      resource: { name: cpu, target: { type: Utilization, averageUtilization: 65 } }
```

- **Probes** apuntan a los endpoints de Actuator; el *startup probe* da margen al arranque de la JVM.
- `server.shutdown=graceful` y `terminationGracePeriodSeconds` suficiente evitan cortar peticiones durante un despliegue.
- **Límite de memoria** coherente con `MaxRAMPercentage`; la JVM respeta los límites del contenedor.
- **Requests de CPU** realistas: el HPA calcula la utilización sobre ellos.

Arranque rápido: GraalVM Native Image y CRaC

- **GraalVM Native Image** (`-Pnative`): arranque en milisegundos y menor memoria; a cambio de builds largos y limitaciones con reflexión.

- **CRaC / checkpoint-restore**: restaura la JVM desde una instantánea para reducir el arranque, con soporte en Spring Boot 3.2+.
- Para la mayoría de servicios, la JVM tradicional con buena configuración es suficiente; adopta lo nativo cuando el escalado a cero o el coste de arranque lo justifiquen.

Pipeline de ejemplo (GitHub Actions)

CI básica

```
name: ci
on: { push: { branches: [main] }, pull_request: {} }
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { distribution: temurin, java-version: 21, cache: maven }
      - run: ./mvnw -B verify                      # tests unitarios e integración
      - name: Build y publicación de la imagen
        if: github.ref == 'refs/heads/main'
        run: |
          ./mvnw -B spring-boot:build-image \
            -Dspring-boot.build-image.imageName=ghcr.io/acme/order-service:${{ github.sha }}
```

29. Escalabilidad y rendimiento en Spring Boot

Hilos de petición: MVC, hilos virtuales o WebFlux

Modelo	Cómo funciona	Ventajas	Atención a
MVC + hilos de plataforma	Un hilo del pool (Tomcat, 200 por defecto) por petición.	Simple; ecosistema bloqueante (JDBC, JPA).	El pool de hilos limita la concurrencia ante E/S lenta.
MVC + hilos virtuales (Java 21+)	Un hilo virtual barato por petición; el bloqueo no retiene un hilo del SO.	Escala la concurrencia sin cambiar el código; ideal para E/S.	Los pools de conexiones (Hikari) siguen limitando; evita `synchronized` largo (pinning en JDK < 24).
WebFlux (reactivo)	Pocos hilos de event loop; E/S no bloqueante.	Streaming, alta concurrencia, backpressure, composición.	Todo el flujo debe ser no bloqueante (R2DBC, WebClient); curva de aprendizaje.

Activar hilos virtuales

```
spring:
  threads:
    virtual:
      enabled: true          # Boot 3.2+ con Java 21+
```

> **BUENA PRÁCTICA**
> Para la mayoría de equipos con JPA/JDBC, **MVC + hilos virtuales** ofrece el mejor equilibrio entre simplicidad y escalabilidad. Reserva WebFlux para gateways, streaming y servicios totalmente reactivos de extremo a extremo.

Pools de conexiones y base de datos

- **Hikari:** `maximum-pool-size` moderado (10-20 por instancia suele bastar). Un pool enorme no acelera la base de datos, la satura.
- **Cuenta el total:** `instancias × pool` debe quedar por debajo del máximo de conexiones de la base de datos, con margen para otros servicios.
- **Consultas:** evita el problema N+1 (`@EntityGraph`, `JOIN FETCH`), pagina siempre y proyecta solo los campos necesarios.
- **Índices** acordes a las consultas reales; revisa los planes de ejecución.
- **Réplicas de lectura:** enruta consultas de solo lectura (`@Transactional(readOnly = true)`) a una réplica con un `AbstractRoutingDataSource`.
- **Batching:** `spring.jpa.properties.hibernate.jdbc.batch_size` para inserciones masivas.

Caché con Spring Cache y Redis

application.yml

```
# dependencia: spring-boot-starter-data-redis y spring-boot-starter-cache
spring:
  cache:
    type: redis
    redis:
      time-to-live: 10m
      key-prefix: "inventory::"
  data:
    redis:
      host: redis
      port: 6379
```

Caché declarativa

```
@Configuration @EnableCaching
class CacheConfig {}

@Service
public class ProductService {
    @Cacheable(cacheNames = "products", key = "#sku", unless = "#result == null")
    public ProductResponse find(String sku) { ... }

    @CacheEvict(cacheNames = "products", key = "#sku")
    public void update(String sku, UpdateProduct cmd) { ... }
}
```

- Define **TTL** en todas las cachés y decide la estrategia de invalidación (por TTL, por evento).
- Cuidado con la **estampida de caché** (muchas peticiones recalculando a la vez): `@Cacheable(sync = true)` o TTL con jitter.
- Caché local (Caffeine) para datos muy calientes y pequeños; Redis cuando debe compartirse entre instancias.
- No caches datos sensibles ni resultados por usuario sin incluir la identidad en la clave.

Trabajo asíncrono y concurrencia controlada

Ejecutores acotados

```
@Configuration
@EnableAsync
class AsyncConfig {
    @Bean(name = "reportExecutor")
    Executor reportExecutor() {
        ThreadPoolTaskExecutor ex = new ThreadPoolTaskExecutor();
        ex.setCorePoolSize(4);
        ex.setMaxPoolSize(8);
        ex.setQueueCapacity(100);                              // cola acotada = backpressure
        ex.setRejectedExecutionHandler(new ThreadPoolExecutor.CallerRunsPolicy());
        ex.setThreadNamePrefix("report-");
        return ex;
    }
}

@Async("reportExecutor")
public CompletableFuture<Report> generate(ReportRequest r) { ... }
```

Escalar con Kafka

- Número de **particiones** ≥ número máximo de consumidores que quieres en paralelo.
- Ajusta `concurrency` del listener (`spring.kafka.listener.concurrency`) a las particiones asignadas por instancia.
- Procesamiento por lotes (`batch-listener`) para aumentar el rendimiento cuando el orden por mensaje no es crítico.

- Escala las réplicas según el **lag** con KEDA.

Rendimiento de la JVM

- Dimensiona el heap con `-XX:MaxRAMPercentage` y deja margen para metaspace, hilos y memoria nativa.
- Usa el recolector por defecto (G1) salvo que midas un problema; ZGC generacional para latencias muy bajas.
- Perfila con **JFR** y **async-profiler** antes de optimizar.
- Reduce el arranque: carga perezosa (`spring.main.lazy-initialization`), eliminar *starters* innecesarios, AOT/CDS.

Checklist de escalabilidad en un servicio Spring

- El servicio es **stateless** (sin sesión HTTP en memoria, sin ficheros locales necesarios).
- Hay **timeouts** en todas las llamadas salientes y pools dimensionados.
- Las consultas están **paginadas e indexadas**, sin N+1.
- Se **cachean** las lecturas calientes con TTL y estrategia de invalidación.
- Lo no urgente va por **mensajería** o tareas asíncronas con colas acotadas.
- Hay **HPA** configurado y probado con carga, y *readiness* correcto para no recibir tráfico antes de tiempo.
- Existen **métricas** para saber cuándo y por qué escalar (saturación, latencia, lag).

30. Checklist final, hoja de ruta y glosario

Checklist de producción para un microservicio Spring Boot

Área	Comprobación
Diseño	Límites claros de dominio; API documentada con OpenAPI; versionado y compatibilidad.
Datos	BD propia; migraciones versionadas; outbox para eventos; consumidores idempotentes.
Resiliencia	Timeouts; retry con backoff solo en operaciones idempotentes; circuit breaker y bulkhead; fallbacks.
Observabilidad	Logs JSON con traceId; métricas RED y de negocio; trazas OTLP; paneles y alertas por SLO.
Seguridad	Resource server con JWT; autorización por scope y recurso; secretos fuera del código; Actuator protegido.
Pruebas	Unitarias, slices, integración con Testcontainers, contratos en CI, pruebas de fallo.
Operación	Probes liveness/readiness; shutdown graceful; límites de recursos; HPA; rollback probado.
Proceso	Pipeline CI/CD; imagen versionada e inmutable; propietario y runbook definidos.

Hoja de ruta de aprendizaje

1. **Sólidos fundamentos:** HTTP/REST, SQL, concurrencia en Java, Spring Boot y Spring Data JPA.
2. **Un servicio completo:** validación, errores con ProblemDetail, Flyway, Actuator y tests con Testcontainers.
3. **Dos servicios hablando:** `WebClient` con timeouts, reintentos y manejo de errores; probarlo con WireMock.
4. **Resiliencia:** Resilience4j (circuit breaker, retry, bulkhead) y pruebas de fallo.
5. **Mensajería:** Kafka con productores, consumidores idempotentes, DLT y outbox.
6. **Gateway, discovery y configuración:** Spring Cloud Gateway y, o bien Eureka/Config, o el equivalente en Kubernetes.
7. **Seguridad:** OAuth2/OIDC con Keycloak y JWT entre servicios.
8. **Observabilidad:** Micrometer, OpenTelemetry, Prometheus y Grafana.
9. **Despliegue:** Docker, Kubernetes, CI/CD y estrategias canary/blue-green.
10. **Patrones avanzados:** sagas, CQRS, event sourcing, contract testing y chaos engineering.

Proyecto de práctica sugerido

Construye la tienda de este documento: `order-service`, `inventory-service` y `notification-service`, más gateway. Comprueba que sabes: (1) hacer que un pedido falle limpiamente cuando inventario tarda más de 3 segundos; (2) abrir el circuit breaker bajo carga y verlo en Grafana; (3) reprocesar un mensaje duplicado sin efectos dobles; (4) seguir una petición completa por su `traceId`; (5) escalar `order-service` a 5 réplicas con HPA bajo una prueba de k6.

Recursos recomendados

- **Libros:** *Building Microservices* (Sam Newman); *Microservices Patterns* (Chris Richardson); *Domain-Driven Design* (Eric Evans); *Designing Data-Intensive Applications* (Martin Kleppmann); *Release It!* (Michael Nygard).
- **Sitios:** microservices.io (catálogo de patrones), martinfowler.com/microservices, 12factor.net, spring.io/guides y la referencia oficial de Spring Cloud.
- **Documentación:** Resilience4j, Project Reactor, Spring Security, Micrometer, OpenTelemetry y Testcontainers.

Glosario

Término	Significado
ACID	Atomicidad, consistencia, aislamiento y durabilidad: garantías de una transacción en una base de datos.
API Gateway	Punto de entrada único que enruta y protege el acceso a los servicios.
At-least-once	Garantía de entrega en la que un mensaje llega una o más veces; exige idempotencia.
Backpressure	Mecanismo para que un consumidor lento regule al productor y no se sature.
Bounded context	Frontera dentro de la cual un modelo de dominio tiene un significado coherente.
BFF	*Backend for Frontend*: gateway específico para un tipo de cliente.
Bulkhead	Aislamiento de recursos para limitar el impacto de un fallo.
Canary	Despliegue gradual a un pequeño porcentaje del tráfico.
CDC	*Change Data Capture*: captura de cambios de una base de datos como eventos.
Circuit Breaker	Patrón que corta llamadas a una dependencia que falla.
CQRS	Separación de los modelos de escritura y de lectura.
DLQ / DLT	Cola o tópico de mensajes que no pudieron procesarse.
Event Sourcing	Persistir la secuencia de eventos como fuente de verdad.
Idempotencia	Propiedad de una operación que produce el mismo efecto si se repite.
Jitter	Aleatoriedad añadida a los tiempos de espera para evitar sincronizaciones.
Mono / Flux	Tipos de Reactor para flujos de 0..1 y 0..N elementos.
mTLS	TLS mutuo: cliente y servidor se autentican con certificados.
Outbox	Tabla de eventos en la misma transacción que los datos, para publicarlos con garantías.
Saga	Transacción de negocio repartida en pasos locales con compensaciones.
Service Discovery	Registro y resolución dinámica de instancias de servicios.
Service Mesh	Capa de infraestructura (sidecars) para tráfico, seguridad y telemetría entre servicios.
SLI / SLO / SLA	Indicador, objetivo y acuerdo de nivel de servicio.
Stateless	Sin estado entre peticiones en la propia instancia.
Strangler Fig	Migración gradual de un sistema legado hacia servicios nuevos.
Trace / Span	Recorrido completo de una petición y cada operación individual dentro de él.

IDEA CLAVE · PARA RECORDAR

Los microservicios resuelven problemas de **escala organizativa y técnica**, a cambio de complejidad distribuida. Empieza simple, mide, divide cuando haya una razón clara, y automatiza la resiliencia, la observabilidad y el despliegue desde el principio. El mejor microservicio es el que no tuviste que crear.